Билеты по дискретной математике

Преподаватель: Востров А. В.

Сверстал: Четвергов Иван | исходный код LaTeX: github

Содержание

1 Множества. Задание множеств. Парадокс Рассела. 4

2 Алгебра подмножеств. Сравнение множеств. Мощность множества. Операции над множествами. 7

3 Разбиения и покрытия. Булеан. Свойства операций над множествами. Диаграммы Эйлера-Венна. 10

4 Счетные и несчетные множества. Теоремы о счетных множествах. Представление множеств в программах. 12

5 Мультимножества. Операции над мультимножествами. 15

6 Отношения (бинарное, n-арное). Связь множеств и упорядоченной пары. Прямое произведение множеств. Композиция отношений. Степень отношения. Ядро отношения. Примеры. 18

7 Свойства отношений. Представление отношений, операций над ними и их свойств матрицами. Примеры. 22

8 Замыкание отношений. Транзитивное и рефлексивное замыкание. Алгоритм Уоршалла. Представление отношений в программах. 24

9 Функциональное отношение. Инъекция, сюръекция, биекция, тотальность. Образы и прообразы. Суперпозиция функций. Представление функций в программах. Примеры. 26

10 Отношение эквивалентности. Классы эквивалентности. Фактор-множества. Ядро функционального отношения и множества уровня. 29

11 Отношение порядка. Минимальные элементы. Верхние и нижние границы. Монотонность. Вполне упорядоченные множества. Диаграммы Хассе. Примеры. 31

12 Характеристические функции множеств и отношений. Функции максимума и минимума. 33

13 Алгебры. Носители и сигнатура. Замыкания и подалгебры. Система образующих. Свойства операций. Примеры. 35

14 Виды морфизмов. Гомоморфизм, изоморфизм. Примеры. 37

15 Алгебры с одной операцией. Полугруппы. Моноиды. Подстановки, композиция подстановок. Две теоремы. Примеры. 39

16 Алгебры с одной операцией. Группы. Группа перестановок. 41

17 Алгебры с двумя операциями. Кольца, поля (3 теоремы). Области целостности. Примеры. 43

18 Конечные алгебры и машинная арифметика. Двоичный смещенный код. «Арифметический круг». Двоичная и логические арифметики. Примеры. 45

19 «Малая» и «большая» конечные арифметики. Примеры. 46

20 Векторное пространство. Линейные комбинации. Базис и размерность. Модули. Примеры. 47

21 Решётки и их виды. Частичный порядок в решётке. Булевы алгебры. Примеры. 49

22 Матроиды. Максимальные независимые подмножества, базисы матроида. Примеры матроидов. Жадный алгоритм. 51

23 Булевы функции. Функции одной и двух переменных, функции трех переменных. Существенные и фиктивные переменные. 53

24 Формулы. Реализация функций формулами, интерпретация формул. Таблицы истинности. Равносильные формулы. 55

25 Алгебра булевых функций и булевы алгебры. Алгебра Линденбаума-Тарского. Эквивалентные преобразования. Разложение Шеннона. 57

26 Двойственность в математике. Принцип двойственности. Представление булевых функций в программах. 59

27 Разложение булевых функций по переменным. Нормальные формы. КНФ, ДНФ, СКНФ, СДНФ. Сокращенные дизъюнктивные формы. 61

28 Минимизация булевых функций. Минимальные и сокращенные ДНФ. Геометрическая интерпретация. 63

29 Полиномы Жегалкина (три способа получения). Линейное представление булевых функций. 65

30 Дифференцирование булевых функций. 67

31 Деревья решений (полные и сокращенные). Синтаксические деревья. Бинарная диаграмма решений. 68

32 Полнота. Замкнутые классы. Полные системы функций и базисы. Теорема Поста. 70

33 Логические формулы, связки и кванторы. Интерпретация, логическое следование. 72

34 Фундаментальные узлы компьютерных систем и двоичные функции. 74

35 Алфавит, слово и язык. Кодирование и его свойства. Методы описания множества сообщений. Алфавитное кодирование. 76

36 Разделимые и префиксные схемы кодирования. Неравенство Макмиллана. Кодирование с минимальной избыточностью. Цена кодирования. Примеры. 78

37 Алгоритм Фано. Оптимальное кодирование. Алгоритм Хаффмана. Дерево Хаффмана. 80

38 Помехоустойчивое кодирование. Кодирование с исправлением ошибок. Возможность исправления всех ошибок. 82

39 Расстояния Левенштейна и Хэмминга. Кодовое расстояние схемы. Мощность кодирования. 84

40 Классические коды коррекции ошибок (коды с повторением, линейные коды). Код Хэмминга. 86

1 Множества. Задание множеств. Парадокс Рассела.

Множества

При построении доступной для рационального анализа картины мира часто ис- пользуется термин **«объект»** для обозначения некой сущности, отделимой от остальных. Выделение объектов — это не более чем произвольный акт нашего сознания.

Элементы и множества Понятие **множества** принадлежит к числу фундаментальных понятий матема- тики. Можно сказать, что множество — это любая определённая **совокупность объектов**. Объекты, из которых составлено множество, называются его **элементами**. Элементы множества различны и отличимы друг от друга.

Если объект x является элементом множества M, то говорят, что x принадлежит M. **Обозначение:** $x \in M$. В противном случае говорят, что x не принадлежит M. **Обозначение:** $x \notin M$.

Примеры

- Множество S страниц в данной книге.
- Множество $\mathbb{N}$ натуральных чисел $\{1, 2, 3, \ldots\}$.
- Множество P простых чисел $\{2, 3, 5, 7, 11, \ldots\}$.
- Множество $\mathbb{Z}$ целых чисел $\{\ldots, -2, -1, 0, 1, 2, \ldots\}$.
- Множество $\mathbb{R}$ вещественных чисел.

Множество, не содержащее элементов, называется **пустым**. **Обозначение:** $\emptyset$.

Множества как объекты могут быть элементами других множеств. Множество, элементами которого являются множества, иногда называют **семейством**.

Совокупность объектов, которая не является множеством, называется **классом**.

Задание множеств

Чтобы задать множество, нужно указать, какие элементы ему принадлежат. Это указание заключают в пару фигурных скобок, оно может иметь одну из следующих основных форм:

перечисление элементов: $M := \{a, b, c, \ldots, z\}$;

характеристический предикат: $M := \{x \mid P(x)\}$;

порождающая процедура: $M := \{x \mid x := f\}$.

При задании множеств перечислением обозначения элементов разделяют за- пятыми. **Характеристический предикат** — это некоторое условие, выраженное в форме логического утверждения или процедуры, возвращающей логическое значение.

Примеры задания множеств

1. $M_9 := \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$. (Перечисление)
2. $M_9 := \{n \mid n \in \mathbb{N} \wedge n < 10\}$. (Характеристический предикат)
3. $M_9 := \{n \mid n := 0;\ \text{for } i \text{ from } 1 \text{ to } 9 \text{ do } n := n + 1;\ \text{yield } n \text{ end for}\}$. (Порождающая процедура)

Парадокс Рассела

Возможность задания множеств характеристическим предикатом зависит от пре- диката. Использование некоторых предикатов для этой цели может приводить к противоречиям.

Рассмотрим множество Y всех множеств, **не** содержащих себя в качестве элемента:

$$Y := \{X \mid X \notin X\}$$

Если множество Y существует, то возникает вопрос: $Y \in Y$?

- **Пусть** $Y \in Y$, **тогда** по определению Y, должно следовать $Y \notin Y$. (**Противоречие**)
- **Пусть** $Y \notin Y$, **тогда** по определению Y (как множества всех множеств, не содержащих себя), должно следовать $Y \in Y$. (**Противоречие**)

Получается неустранимое логическое противоречие, известное как **парадокс Рассела**.

Способы избежать парадокса Рассела

1. **Ограничить характеристические предикаты:** Предикат должен быть вида $P(x) = x \in A \wedge Q(x)$, где A — известное, заведомо существующее множество (**универсум**). Используют обозначение $\{x \in A \mid Q(x)\}$. Для Y универсум не указан, а потому Y множеством не является.

2. **Теория типов:** Объекты имеют тип 0, множества элементов типа 0 имеют тип 1, множества элементов типа 0 и 1 — тип 2 и т. д. Y не имеет типа и множеством не является.

3. **Явный запрет принадлежности множества самому себе:** $X \in X$ — недопустимый предикат. При аксиоматическом построении теории множеств соответствующая аксиома называется **аксиомой регулярности**.

2 Алгебра подмножеств. Сравнение множеств. Мощность множества. Операции над множествами.

Сравнение множеств

Для конструирования новых множеств из имеющихся определяются **операции над множествами**.

Подмножество и Надмножество Множество A **содержится** в множестве B (или B **включает** A), если каждый элемент множества A есть элемент множества B:

$$A \subset B \overset{\text{def}}{\iff} \forall x(x \in A \implies x \in B).$$

В этом случае A называется **подмножеством** B, B — **надмножеством** A. По определению, $\forall M(\emptyset \subset M)$.

Равенство множеств Два множества **равны**, если они являются подмножествами друг друга:

$$A = B \iff (A \subset B \wedge B \subset A).$$

Свойства включения множеств (Теорема) Включение множеств обладает следующими свойствами:

1. Рефлексивность: $\forall A(A \subset A)$.
2. Антисимметричность: $\forall A, B(A \subset B \wedge B \subset A \implies A = B)$.
3. Транзитивность: $\forall A, B, C(A \subset B \wedge B \subset C \implies A \subset C)$.

Мощность множества

Равномощные множества Говорят, что между множествами A и B установлено **взаимно-однозначное соответствие** (или они **изоморфны**, $A \sim B$), если:

- Каждому элементу $a \in A$ соответствует один и только один элемент $b \in B$.
- Для каждого элемента $b \in B$ соответствует один и только один элемент $a \in A$.

Если $a \in A$ соответствует $b \in B$, обозначают $a \mapsto b$.

Пример: Соответствие $n \mapsto 2n$ устанавливает взаимно-однозначное соответствие между множеством натуральных чисел $\mathbb{N}$ и множеством чётных натуральных чисел $2\mathbb{N}$. ($\mathbb{N} \sim 2\mathbb{N}$)

Одинаковая мощность Если между двумя множествами A и B может быть установлено взаимно-однозначное соответствие, то говорят, что множества имеют **одинаковую мощность** или **равномощны**, и записывают это как $|A| = |B|$.

$$|A| = |B| \iff A \sim B.$$

Свойства равномощности (Теорема) Равномощность множеств обладает следующими свойствами:

1. Рефлексивность: $\forall A(|A| = |A|)$.
2. Симметричность: $\forall A, B(|A| = |B| \implies |B| = |A|)$.
3. Транзитивность: $\forall A, B, C(|A| = |B| \wedge |B| = |C| \implies |A| = |C|)$.

Конечные и бесконечные множества

Конечное множество Множество A называется **конечным**, если у него нет равномощного **собственного** подмножества ($B \subset A$ и $B \neq A$):

$$\forall B((B \subset A \wedge |B| = |A|) \implies B = A).$$

Обозначение: $|A| < \infty$.

Бесконечное множество Множество A называется **бесконечным**, если оно равномощно некоторому своему собственному подмножеству:

$$\exists B(B \subset A \wedge |B| = |A| \wedge B \neq A).$$

Обозначение: $|A| = \infty$.

Пример: Множество $\mathbb{N}$ бесконечно, $|\mathbb{N}| = \infty$, так как оно равномощно своему собственному подмножеству чётных чисел $2\mathbb{N}$.

Теорема: Множество, имеющее бесконечное подмножество, бесконечно:

$$(B \subset A \wedge |B| = \infty) \implies (|A| = \infty).$$

Мощность конечного множества **Теорема:** Любое непустое конечное множество равномощно некоторому отрезку натурального ряда:

$$\forall A(A \neq \emptyset \wedge |A| < \infty \implies \exists k \in \mathbb{N}(|A| = |\{1, \ldots, k\}|)).$$

Операции над множествами

Обычно рассматриваются следующие операции над множествами:

Объединение (Union): $A \cup B \stackrel{\text{def}}{=} \{x \mid x \in A \vee x \in B\}$

Пересечение (Intersection): $A \cap B \stackrel{\text{def}}{=} \{x \mid x \in A \wedge x \in B\}$

Разность (Difference): $A \setminus B \stackrel{\text{def}}{=} \{x \mid x \in A \wedge x \notin B\}$

Симметрическая разность (Symmetric Difference): $A \Delta B \stackrel{\text{def}}{=} (A \cup B) \setminus (A \cap B) = \{x \mid (x \in A \wedge x \notin B) \vee (x \notin A \wedge x \in B)\}$

Дополнение (Complement): $\overline{A} \stackrel{\text{def}}{=} \{x \mid x \notin A\}$

ЗАМЕЧАНИЕ: Операция дополнения $\overline{A}$ определена только при заданном **универсуме** U: $\overline{A} = U \setminus A$.

Формулы для мощностей (для конечных множеств)

- $|\mathrm{A} \cup \mathrm{B}| = |\mathrm{A}| + |\mathrm{B}| - |\mathrm{A} \cap \mathrm{B}|$
- $|\mathrm{A} \setminus \mathrm{B}| = |\mathrm{A}| - |\mathrm{A} \cap \mathrm{B}|$
- $|\mathrm{A} \Delta \mathrm{B}| = |\mathrm{A}| + |\mathrm{B}| - 2|\mathrm{A} \cap \mathrm{B}|$

3 Разбиения и покрытия. Булеан. Свойства операций над множествами. Диаграммы Эйлера-Венна.

Разбиения и покрытия

Пусть $\mathcal{E} = \{E_i\}_{i \in I}$ — некоторое **семейство подмножеств** множества M, $E_i \subset M$.

Покрытие Семейство $\mathcal{E}$ называется **покрытием** множества M, если каждый элемент $x \in M$ принадлежит хотя бы одному из множеств E_i:

$$\forall x \in M (\exists i \in I (x \in E_i)).$$

Дизъюнктное семейство Семейство $\mathcal{E}$ называется **дизъюнктным**, если элементы этого семейства **попарно не пересекаются**:

$$\forall i, j \in I (i \neq j \implies E_i \cap E_j = \emptyset).$$

Разбиение **Дизъюнктное покрытие** называется **разбиением** множества M. Элементы разбиения (подмножества E_i) часто называют **блоками** разбиения.

Пример: Пусть $M = \{1, 2, 3\}$.

- $\{\{1, 2\}, \{2, 3\}, \{3, 1\}\}$ — покрытие, но не разбиение (есть пересечения).
- $\{\{1\}, \{2\}, \{3\}\}$ — разбиение (и покрытие).
- $\{\{1\}, \{2\}\}$ — дизъюнктное, но не покрытие, следовательно не разбиение.

Булеан (Множество подмножеств)

Определение Булеана **Множество всех подмножеств** множества M называется **булеаном** множества M и обозначается $\mathbf{2^M}$ или $\mathcal{P}(M)$:

$$\mathbf{2^M} \stackrel{\text{def}}{=} \{A \mid A \subset M\}.$$

Теорема о мощности Булеана Если множество M конечно, то мощность его булеана равна 2 в степени мощности M:

$$|\mathbf{2^M}| = 2^{|M|}.$$

Алгебра подмножеств Множество всех подмножеств универсума U, снабженное операциями пересечения, объединения, разности и дополнения, образует **алгебру подмножеств** множества U.

Свойства операций над множествами

Пусть задан универсум U. Тогда $\forall A, B, C \subset U$ выполняются следующие равенства (где $\overline{A}$ — дополнение A до U, т.е. $U \setminus A$):

1. **Идемпотентность:**
$$A \cup A = A, \quad A \cap A = A$$

2. **Коммутативность:**
$$A \cup B = B \cup A, \quad A \cap B = B \cap A$$

3. **Ассоциативность:**
$$A \cup (B \cup C) = (A \cup B) \cup C, \quad A \cap (B \cap C) = (A \cap B) \cap C$$

4. **Дистрибутивность:**
$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$
$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

5. **Поглощение:**
$$(A \cap B) \cup A = A, \quad (A \cup B) \cap A = A$$

6. **Свойства нуля ($\emptyset$):**
$$A \cup \emptyset = A, \quad A \cap \emptyset = \emptyset$$

7. **Свойства единицы (U):**
$$A \cup U = U, \quad A \cap U = A$$

8. **Инволютивность (двойное дополнение):**
$$\overline{\overline{A}} = A$$

9. **Законы де Моргана:**
$$\overline{A \cap B} = \overline{A} \cup \overline{B}, \quad \overline{A \cup B} = \overline{A} \cap \overline{B}$$

10. **Свойства дополнения:**
$$A \cup \overline{A} = U, \quad A \cap \overline{A} = \emptyset$$

11. **Выражение для разности:**
$$A \setminus B = A \cap \overline{B}$$

4 Счетные и несчетные множества. Теоремы о счетных множествах. Представление множеств в программах.

Счетные и Несчетные множества.

Счетное множество Множество A называется **счетным**, если оно равномощно множеству натуральных чисел $\mathbb{N}$ (т.е., $|A| = |\mathbb{N}| = \aleph_0$). Элементы счетного множества могут быть перечислены в последовательность $a_1, a_2, a_3, \ldots$.

Несчетное множество Множество B называется **несчетным**, если оно не является ни конечным, ни счетным (т.е., $|B| > |\mathbb{N}|$). **Пример:** Множество вещественных чисел $\mathbb{R}$.

Теорема Кантора Мощность булеана любого множества A всегда строго больше мощности самого множества:

$$|\mathbf{2^A}| > |A|.$$

Это доказывает, что несчетные множества существуют.

Теорема Кантора-Бернштейна Если существует инъекция (вложение) множества A в B и инъекция B в A, то множества A и B равномощны:

$$(A \hookrightarrow B \wedge B \hookrightarrow A) \implies |A| = |B|.$$

Представление множеств в программах

Определение представления **Представить объект** (множество) в программе — значит описать:

1. **Структуру данных**, используемую для хранения информации о множестве (например, о принадлежности элементов).

2. **Алгоритмы** над этими структурами, реализующие операции над множествами (объединение, пересечение и т.д.).

Выбор представления (например, битовые шкалы, списки, хеш-таблицы) зависит от особенностей множества, состава и частоты использования операций.

Битовые шкалы (Set as Bit Array)

Пусть задан конечный универсум $U = \{u_1, \ldots, u_n\}$, где $|U| \leq n$ (разрядности компьютера). Подмножество $A \subset U$ представляется кодом (**битовой шкалой**) C : array[1..n] of 0..1, где:

$$C[i] = 1 \iff u_i \in A.$$

Операции над битовыми шкалами Операции над небольшими множествами в этом представлении выполняются весьма эффективно (часто одной машинной командой):

- **Пересечение** ($A \cap B$): поразрядное логическое `И` кодов C_A и C_B.
- **Объединение** ($A \cup B$): поразрядное логическое `ИЛИ` кодов C_A и C_B.
- **Дополнение** ($\overline{A}$): поразрядная `ИНВЕРСИЯ` кода C_A.

ЗАМЕЧАНИЕ: Если $|U|$ превосходит размер машинного слова, используются **массивы битовых шкал** (массивы машинных слов), а операции реализуются циклами по элементам массива.

Генерация всех подмножеств универсума

Для генерации всех 2^n подмножеств n-элементного множества $\{a_1, \ldots, a_n\}$ используется соответствие: каждое целое число i от 0 до $2^n - 1$ представляет одно подмножество.

Алгоритм 1.1 Генерация всех подмножеств **Вход:** $n \geq 0$ — мощность множества. **Выход:** последовательность кодов подмножеств i.

```
for i from 0 to 2^n - 1 do
    yield i  { код очередного подмножества }
end for
```

Алгоритм построения бинарного кода Грея

Алгоритм генерирует последовательность всех подмножеств так, что каждое следующее подмножество получается из предыдущего **изменением в точности одного элемента** (удалением или добавлением).

Алгоритм 1.2 Построение бинарного кода Грея **Вход:** $n \geq 0$ — мощность множества. **Выход:** последовательность кодов подмножеств B.

```
B : array [1..n] of 0..1  { битовая шкала }
for i from 1 to n do
    B[i] := 0 { инициализация }
end for

yield B { пустое множество }
for i from 1 to 2^n - 1 do
    p := Q(i) { номер элемента для изменения }
    B[p] := 1 - B[p] { инверсия бита }
    yield B { очередное подмножество }
end for
```

Функция Q Функция $Q(i)$ возвращает номер разряда p, подлежащего изменению. $Q(i)$ определяется как **количество нулей на конце двоичной записи числа** i, увеличенное на 1.

Алгоритм 1.3 Функция Q определения изменяемого разряда **Вход:** i — номер подмножества. **Выход:** номер изменяемого разряда q.

```
q := 1; j := i
while j четно do
    j := j / 2
    q := q + 1
end while
return q
```

5 Мультимножества. Операции над мультимножествами.

Мультимножества

В отличие от обычного множества, где все элементы различны и входят ровно один раз, **мультимножество** допускает, что элементы могут входить в совокупность **по несколько раз**.

Определение Пусть $X = \{x_1, \ldots, x_n\}$ — некоторое (конечное) множество, и пусть $\alpha_1, \ldots, \alpha_n$ — неотрицательные целые числа.

Мультимножеством $\mathcal{X}$ над множеством X называется совокупность элементов множества X, в которую элемент x_i входит α_i раз, $\alpha_i \geq 0$.

Обозначения мультимножеств Мультимножество $\mathcal{X}$ обозначается одним из следующих способов:

- С использованием показателей (степеней):

$$\mathcal{X} = [x_1^{\alpha_1}, \ldots, x_n^{\alpha_n}]$$

- Перечислением элементов (где x_i повторяется α_i раз):

$$\mathcal{X} = (x_1, \ldots, x_1, \ldots, x_n, \ldots, x_n)$$

- С использованием пары (показатель, элемент):

$$\mathcal{X} = (\alpha_1(x_1), \ldots, \alpha_n(x_n))$$

Пример: Пусть $X = \{a, b, c\}$. Тогда $\mathcal{X} = [a^0 b^3 c^4] = (b, b, b, c, c, c, c) = (0(a), 3(b), 4(c))$.

Основные характеристики Пусть $\mathcal{X} = (\alpha_1(x_1), \ldots, \alpha_n(x_n))$ — мультимножество над $X = \{x_1, \ldots, x_n\}$.

- **Показатель (кратность) элемента x_i:** Число α_i.
- **Носитель мультимножества:** Множество X, над которым определено $\mathcal{X}$.
- **Мощность мультимножества $|\mathcal{X}|$:** Общее число элементов с учетом кратности:

$$m = \alpha_1 + \alpha_2 + \cdots + \alpha_n.$$

- **Состав мультимножества:** Множество элементов, имеющих положительный показатель (кратность $\alpha_i > 0$):

$$X' = \{x_i \in X \mid \alpha_i > 0\}.$$

ЗАМЕЧАНИЕ: Элементы мультимножества, равно как и элементы множества, считаются **неупорядоченными**.

Операции над мультимножествами

Пусть $\mathcal{A} = (\alpha_i(x_i))$ и $\mathcal{B} = (\beta_i(x_i))$ — два мультимножества над одним носителем $X = \{x_1, \ldots, x_n\}$, где α_i и β_i — кратности элемента x_i в $\mathcal{A}$ и $\mathcal{B}$ соответственно. U — универсум с кратностью u_i для элемента x_i.

Логические (Теоретико-множественные) операции

В основе этих операций лежат функции max и min для кратностей:

1. **Объединение ($\mathcal{A} \cup \mathcal{B}$):** Кратность элемента x_i равна **максимуму** из кратностей:
$$\mathcal{C} = \mathcal{A} \cup \mathcal{B} \implies \gamma_i = \max(\alpha_i, \beta_i).$$

2. **Пересечение ($\mathcal{A} \cap \mathcal{B}$):** Кратность элемента x_i равна **минимуму** из кратностей:
$$\mathcal{C} = \mathcal{A} \cap \mathcal{B} \implies \gamma_i = \min(\alpha_i, \beta_i).$$

3. **Разность ($\mathcal{A} \setminus \mathcal{B}$):** Кратность элемента x_i равна **разности кратностей**, ограниченной снизу нулем:
$$\mathcal{C} = \mathcal{A} \setminus \mathcal{B} \implies \gamma_i = \max(\alpha_i - \beta_i, 0).$$

4. **Симметрическая разность ($\mathcal{A} \Delta \mathcal{B}$):** Кратность элемента x_i равна **модулю разности кратностей**:
$$\mathcal{C} = \mathcal{A} \Delta \mathcal{B} \implies \gamma_i = |\alpha_i - \beta_i|.$$

5. **Дополнение ($\overline{\mathcal{A}}$):** Дополнение относительно универсума $\mathcal{U}$:
$$\mathcal{C} = \overline{\mathcal{A}} = \mathcal{U} \setminus \mathcal{A} \implies \gamma_i = \max(u_i - \alpha_i, 0).$$

Арифметические операции

Эти операции используют арифметические правила для кратностей, часто с ограничением сверху кратностью в универсуме (u_i).

1. **Арифметическая сумма ($\mathcal{A}+\mathcal{B}$):** Кратность элемента x_i равна **сумме кратностей**, ограниченной u_i:

$$\mathcal{C} = \mathcal{A} + \mathcal{B} \implies \gamma_i = \min(\alpha_i + \beta_i, u_i).$$

2. **Арифметическая разность ($\mathcal{A} - \mathcal{B}$):** Кратность элемента x_i равна **разности кратностей**, ограниченной снизу нулем:

$$\mathcal{C} = \mathcal{A} - \mathcal{B} \implies \gamma_i = \max(\alpha_i - \beta_i, 0).$$

3. **Арифметическое произведение ($\mathcal{A}\times\mathcal{B}$):** Кратность элемента x_i равна **произведению кратностей**, ограниченному u_i:

$$\mathcal{C} = \mathcal{A} \times \mathcal{B} \implies \gamma_i = \min(\alpha_i \cdot \beta_i, u_i).$$

4. **Арифметическое деление ($\mathcal{A} \div \mathcal{B}$):** Кратность элемента x_i равна **целой части от деления кратностей**:

$$\mathcal{C} = \mathcal{A} \div \mathcal{B} \implies \gamma_i = \begin{cases} \lfloor \alpha_i/\beta_i \rfloor, & \text{если } \beta_i \neq 0 \\ 0, & \text{если } \beta_i = 0 \end{cases}$$

(С учетом ограничения универсума: $\gamma_i = \min(\lfloor \alpha_i/\beta_i \rfloor, u_i)$ при $\beta_i \neq 0$).

6 Отношения (бинарное, n-арное). Связь множеств и упорядоченной пары. Прямое произведение множеств. Композиция отношений. Степень отношения. Ядро отношения. Примеры.

Упорядоченные пары и наборы

Упорядоченная пара Для объектов a и b **упорядоченная пара** обозначается (a, b). **Равенство** упорядоченных пар:

$$(a, b) = (c, d) \overset{\text{def}}{\iff} a = c \wedge b = d.$$

В общем случае $(a, b) \neq (b, a)$.

Упорядоченный набор (n-ка, кортеж) Упорядоченный набор из n элементов обозначается $(a_1, \ldots, a_n)$. Набор может быть определен рекурсивно: $(a_1, \ldots, a_n) \overset{\text{def}}{=} ((a_1, \ldots, a_{n-1}), a_n)$. **Длина** набора: $|(a_1, \ldots, a_n)| = n$.

Теорема о равенстве наборов: Два набора одной длины равны, если равны их соответствующие элементы:

$$\forall n \geq 1 : (a_1, \ldots, a_n) = (b_1, \ldots, b_n) \iff \forall i \in \{1, \ldots, n\}(a_i = b_i).$$

Прямое произведение множеств

Прямое (декартово) произведение двух множеств Для множеств A и B **прямым произведением** $A \times B$ называется множество всех упорядоченных пар, в которых первый элемент принадлежит A, а второй принадлежит B:

$$A \times B \overset{\text{def}}{=} \{(a, b) \mid a \in A \wedge b \in B\}.$$

Теорема о мощности: Для конечных множеств A и B мощность произведения равна произведению мощностей: $|A \times B| = |A| \cdot |B|$.

n-кратное прямое произведение Прямое произведение n множеств $A_1, \ldots, A_n$ — это множество наборов (кортежей):

$$A_1 \times \cdots \times A_n \overset{\text{def}}{=} \{(a_1, \ldots, a_n) \mid a_1 \in A_1 \wedge \cdots \wedge a_n \in A_n\}.$$

Степень множества **Степенью** множества A называется его n-кратное прямое произведение самого на себя:

$$A^n \stackrel{\text{def}}{=} \underbrace{A \times \cdots \times A}_{n \text{ раз}}.$$

Следствие: $|A^n| = |A|^n$.

Отношения (Бинарное и n-арное)

Бинарное отношение **Бинарным отношением** $\mathcal{R}$ между множествами A и B называется тройка (A, B, R), где R — **подмножество прямого произведения** $A \times B$:

$$R \subset A \times B.$$

- R называется **графиком отношения**.
- A — **область отправления** (домен), B — **область прибытия** (кодомен).
- Если $A = B$ ($R \subset A^2$), то $\mathcal{R}$ называется **отношением на множестве** A.

Для бинарных отношений используется **инфиксная форма записи**: $a\mathcal{R}b \stackrel{\text{def}}{\Longleftrightarrow} (a, b) \in R$.

Характеристики бинарного отношения Пусть $R \subset A \times B$.

- **Область определения** ($\mathrm{Dom}\ R$): Множество элементов A, участвующих в парах отношения:

$$\mathrm{Dom}\ R \stackrel{\text{def}}{=} \{a \in A \mid \exists b \in B((a, b) \in R)\}.$$

- **Область значений** ($\mathrm{Im}\ R$): Множество элементов B, участвующих в парах отношения:

$$\mathrm{Im}\ R \stackrel{\text{def}}{=} \{b \in B \mid \exists a \in A((a, b) \in R)\}.$$

- **Обратное отношение** ($R^{-1} \subset B \times A$):

$$R^{-1} \stackrel{\text{def}}{=} \{(b, a) \mid (a, b) \in R\}.$$

- **Дополнение отношения** ($\overline{R} \subset A \times B$):

$$\overline{R} \stackrel{\text{def}}{=} \{(a, b) \mid (a, b) \notin R\} = (A \times B) \setminus R.$$

n-арное отношение **n-местное** (n-арное) отношение $\mathcal{R}$ — это подмножество прямого произведения n множеств:

$$R \subset A_1 \times A_2 \times \cdots \times A_n \stackrel{\text{def}}{=} \{(a_1, \ldots, a_n) \mid a_i \in A_i\}.$$

Композиция отношений и Степень

Композиция отношений Пусть $R_1 \subset A \times C$ и $R_2 \subset C \times B$. **Композицией** отношений R_1 и R_2 называется отношение $R = R_1 \circ R_2 \subset A \times B$, определяемое:

$$a(R_1 \circ R_2)b \stackrel{\text{def}}{\iff} \exists c \in C(aR_1c \wedge cR_2b).$$

Теорема: Композиция отношений **ассоциативна**:

$$\forall R_1 \subset A \times B, R_2 \subset B \times C, R_3 \subset C \times D : (R_1 \circ R_2) \circ R_3 = R_1 \circ (R_2 \circ R_3).$$

ЗАМЕЧАНИЕ: Композиция отношений, в общем случае, **не коммутативна** ($R_1 \circ R_2 \neq R_2 \circ R_1$).

Степень отношения Пусть R — отношение на множестве A ($R \subset A^2$). **Степенью** отношения R называется его n-кратная композиция с самим собой:

$$R^n \stackrel{\text{def}}{=} \underbrace{R \circ R \circ \cdots \circ R}_{n \text{ раз}}.$$

По определению: $R^0 \stackrel{\text{def}}{=} I$ (тождественное отношение), $R^1 \stackrel{\text{def}}{=} R$, и $R^n \stackrel{\text{def}}{=} R^{n-1} \circ R$.

Ядро отношения

Определение Ядра Пусть $R \subset A \times B$ — отношение между множествами A и B. **Ядром отношения** R называется **композиция** отношения R с его обратным отношением R^{-1}:

$$\ker R \stackrel{\text{def}}{=} R \circ R^{-1} \subset A^2.$$

Другими словами, a_1 находится в отношении $\ker R$ с a_2 тогда и только тогда, когда существует хотя бы один общий элемент $b \in B$, связанный с обоими:

$$a_1 \ker R a_2 \stackrel{\text{def}}{\iff} \exists b \in B(a_1Rb \wedge a_2Rb).$$

Ядро отношения R между A и B является **отношением на** A: $R \subset A \times B \implies \ker R \subset A^2$.

Свойства Ядра (Теорема) Ядро любого отношения рефлексивно и симметрично на своей области определения ($\mathrm{Dom}\ R$):

1. **Рефлексивность:** $\forall a \in \mathrm{Dom}\ R(a \ker Ra)$.

2. **Симметричность:** $\forall a, b \in \mathrm{Dom}\ R(a \ker Rb \implies b \ker Ra)$.

Примеры

1. Пусть $R \subset \mathrm{U} \times \mathbf{2}^{\mathrm{U}}$ — отношение принадлежности $\in$. Ядро отношения $\in$ универсально: $\ker(\in) = \mathrm{U} \times \mathrm{U}$.

2. Отношение N_1 на $\mathbb{Z}$: $N_1 = \{(n, m) \mid |n - m| \leq 1\}$. Ядром этого отношения является отношение N_2 (находиться на расстоянии не более 2):

$$\ker N_1 = N_1 \circ N_1^{-1} = N_2 = \{(n, m) \mid |n - m| \leq 2\}.$$

3. Ядром отношения «тесного включения» ($X \subset Y \iff X \subset Y \wedge |X| + 1 = |Y|$) является отношение «отличаться не более чем одним элементом»:

$$\ker(\subset_1) = \{(X, Y) \in \mathbf{2}^{\mathrm{U}} \times \mathbf{2}^{\mathrm{U}} \mid |X| = |Y| \wedge |X \cap Y| \geq |X| - 1\}.$$

ЗАМЕЧАНИЕ: Ядро отношения $\ker R$ всегда является **отношением толерантности** на $\mathrm{Dom}\ R$, так как оно рефлексивно и симметрично.

7 Свойства отношений. Представление отношений, операций над ними и их свойств матрицами. Примеры.

Свойства бинарных отношений

Пусть R — отношение на множестве A ($R \subset A^2$), а $a, b, c \in A$. Отношение R называется:

Рефлексивным: если $\forall a \in A : (aRa)$.

Антирефлексивным: если $\forall a \in A : (\neg aRa)$.

Симметричным: если $\forall a, b \in A : (aRb \implies bRa)$.

Антисимметричным: если $\forall a, b \in A : (aRb \wedge bRa \implies a = b)$.

Транзитивным: если $\forall a, b, c \in A : (aRb \wedge bRc \implies aRc)$.

Линейным (Полным): если $\forall a, b \in A : (a = b \vee aRb \vee bRa)$.

Свойства, выраженные через операции (Теорема) Пусть $R \subset A^2$. I — тождественное отношение на A, $U = A^2$ — универсальное отношение.

1. R рефлексивно $\iff I \subset R$.
2. R симметрично $\iff R = R^{-1}$ (отношение равно своему обратному).
3. R транзитивно $\iff R \circ R \subset R$ (содержится в самом отношении).
4. R антисимметрично $\iff R \cap R^{-1} \subset I$.
5. R антирефлексивно $\iff R \cap I = \emptyset$.
6. R линейно $\iff R \cup I \cup R^{-1} = U$.

Матричное представление отношений

Пусть R — отношение на конечном множестве $A = \{a_1, \ldots, a_n\}$, $|A| = n$. Отношение R представляется **булевой матрицей R** : array$[1..n, 1..n]$ of 0..1:

$$\mathbf{R}[i, j] = 1 \iff a_i R a_j.$$

Матричное представление свойств Для булевой матрицы $\mathbf{R}$ отношения R на A:

- R **рефлексивно** $\iff$ Главная диагональ $\mathbf{R}$ состоит из единиц ($\forall i$: $\mathbf{R}[i,i] = 1$).
- R **антирефлексивно** $\iff$ Главная диагональ $\mathbf{R}$ состоит из нулей ($\forall i$: $\mathbf{R}[i,i] = 0$).
- R **симметрично** $\iff$ $\mathbf{R} = \mathbf{R}^T$ (матрица симметрична).
- R **антисимметрично** $\iff$ Для $i \neq j$: $\mathbf{R}[i,j] = 1 \implies \mathbf{R}[j,i] = 0$.
- R **транзитивно** $\iff$ $\mathbf{R} \cdot \mathbf{R} \subset \mathbf{R}$ (где $\cdot$ — булево умножение матриц).

Операции над отношениями через матрицы (Теоремы) Пусть $\mathbf{R}_1$ и $\mathbf{R}_2$ — булевы матрицы отношений R_1 и R_2.

1. **Обратное отношение (R^{-1}):** Матрица обратного отношения равна **транспонированной** матрице:
$$\mathbf{R^{-1}} = \mathbf{R}^T.$$
2. **Дополнение отношения ($\overline{R}$):** Матрица дополнения получается **инвертированием** всех элементов (замена $1 \to 0$, $0 \to 1$):
$$\overline{\mathbf{R}}[i,j] = 1 - \mathbf{R}[i,j].$$
3. **Объединение ($R_1 \cup R_2$):** Матрица объединения равна **булевой дизъюнкции** (логическое $\vee$) матриц:
$$\mathbf{R_1} \cup \mathbf{R_2} = \mathbf{R}_1 \vee \mathbf{R}_2.$$
4. **Пересечение ($R_1 \cap R_2$):** Матрица пересечения равна **булевой конъюнкции** (логическое $\wedge$) матриц:
$$\mathbf{R_1} \cap \mathbf{R_2} = \mathbf{R}_1 \wedge \mathbf{R}_2.$$
5. **Композиция ($R_1 \circ R_2$):** Матрица композиции равна **булевому произведению** матриц:
$$\mathbf{R_1} \circ \mathbf{R_2} = \mathbf{R}_1 \cdot \mathbf{R}_2.$$
(Булево произведение: $(\mathbf{R}_1 \cdot \mathbf{R}_2)[i,j] = \bigvee_{k=1}^{n} (\mathbf{R}_1[i,k] \wedge \mathbf{R}_2[k,j])$).

8 Замыкание отношений. Транзитивное и рефлексивное замыкание. Алгоритм Уоршалла. Представление отношений в программах.

Замыкание отношений

Пусть R и R' — отношения на множестве M. Отношение R' называется **замыканием** R относительно свойства $\mathcal{C}$, если:

1. R' обладает свойством $\mathcal{C}$ ($\mathcal{C}(R')$).
2. R' является надмножеством R ($R \subset R'$).
3. R' является **наименьшим** таким объектом: если $\mathcal{C}(R'')$ и $R \subset R''$, то $R' \subset R''$.

Транзитивное и Рефлексивное замыкание

Для отношения R на множестве M вводятся объединения его положительных и неотрицательных степеней (композиций):

Транзитивное замыкание (R^+) **Транзитивное замыкание** R^+ — это объединение всех **положительных** степеней R:

$$R^+ \stackrel{\text{def}}{=} \bigcup_{i=1}^{\infty} R^i = R^1 \cup R^2 \cup R^3 \cup \cdots$$

Теорема: R^+ является **наименьшим транзитивным надмножеством** R. R^+ содержит пару (a, b) тогда и только тогда, когда существует путь из a в b по отношению R (длиной ≥ 1).

Рефлексивно-транзитивное замыкание (R^*) **Рефлексивно-транзитивное замыкание** R^* — это объединение всех **неотрицательных** степеней R:

$$R^* \stackrel{\text{def}}{=} \bigcup_{i=0}^{\infty} R^i = R^0 \cup R^1 \cup R^2 \cup \cdots$$

Поскольку $R^0 = I$ (тождественное отношение), то $R^* = R^+ \cup I$. **Теорема:** R^* является **наименьшим рефлексивным и транзитивным надмножеством** R. R^* содержит пару (a, b) тогда и только тогда, когда существует путь из a в b по R (длиной ≥ 0).

Алгоритм Уоршалла

Алгоритм Уоршалла (Warshall's Algorithm) используется для эффективного вычисления **транзитивного замыкания** R^+ отношения R на множестве M, $|M| = n$. Он имеет сложность $O(n^3)$.

Алгоритм работает с булевой матрицей $\mathbf{R}$ отношения.

Алгоритм 1.11 Вычисление транзитивного замыкания **Вход:** Булева матрица отношения $\mathbf{R} : [1..n, 1..n]$. **Выход:** Булева матрица транзитивного замыкания $\mathbf{T}$ (т.е., $\mathbf{R}^+$).

```
T := R

for k from 1 to n do
    for i from 1 to n do
        for j from 1 to n do
            T[i, j] := T[i, j] OR (T[i, k] AND T[k, j])
        end for
    end for
end for
```

Смысл: На k-й итерации внешний цикл гарантирует, что если есть путь из i в j, проходящий только через промежуточные вершины с номерами $\leq k$, то $\mathbf{T}[i, j]$ устанавливается в 1.

Представление отношений в программах

Булева матрица (Матрица смежности) Отношение R на множестве $A = \{a_1, \ldots, a_n\}$ представляется **булевой матрицей** $\mathbf{R}[n \times n]$, где:

$$\mathbf{R}[i, j] = 1 \iff (a_i, a_j) \in R.$$

Матричное представление эффективно для:

- **Проверки свойств:** Симметричность ($\mathbf{R} = \mathbf{R}^T$), Рефлексивность (диагональ из 1).
- **Операций:** Композиция $R_1 \circ R_2$ — булево произведение $\mathbf{R}_1 \cdot \mathbf{R}_2$.
- **Замыкания:** Алгоритм Уоршалла работает непосредственно с этой матрицей.

ЗАМЕЧАНИЕ: Для больших разреженных отношений часто используются другие представления, например, **списки смежности** (см. Графы).

9 Функциональное отношение. Инъекция, сюръекция, биекция, тотальность. Образы и прообразы. Суперпозиция функций. Представление функций в программах. Примеры.

Функциональное отношение (Функция)

Определение Функция f из A в B ($f : A \to B$) — это **бинарное отношение** $f \subset A \times B$, обладающее свойством **однозначности** (функциональности):

$$\forall a \in A, b, c \in B : ((a, b) \in f \wedge (a, c) \in f \implies b = c).$$

То есть, каждому элементу области отправления соответствует не более одного элемента области прибытия.

Терминология

- A — **Область отправления** (Домен).
- B — **Область прибытия** (Кодомен).
- **Область определения** ($\mathrm{Dom}\ f$) — множество $a \in A$, для которых $f(a)$ определено.
- **Область значений** ($\mathrm{Im}\ f$) — множество $b \in B$, являющихся значениями функции: $\mathrm{Im}\ f = \{b \in B \mid \exists a \in A(b = f(a))\}$.

Тотальность и Частичность

- Функция $f : A \to B$ называется **тотальной**, если $\mathrm{Dom}\ f = A$. (Определена для всех элементов A).
- Функция $f : A \to B$ называется **частичной**, если $\mathrm{Dom}\ f \subsetneq A$.

Преобразование: Тотальная функция $f : M \to M$ называется **преобразованием** над M.

Инъекция, Сюръекция и Биекция

Пусть $f : A \to B$ — тотальная функция.

Инъективная (Инъекция): Если разным аргументам соответствуют разные значения:

$$f(a_1) = f(a_2) \implies a_1 = a_2.$$

Сюръективная (Сюръекция): Если область значений совпадает с областью прибытия (каждый элемент B является образом хотя бы одного элемента A):

$$\forall b \in B(\exists a \in A(b = f(a))).$$

Биективная (Биекция): Если функция является одновременно **инъективной** и **сюръективной**. Биекция также называется **взаимно-однозначным соответствием**.

Образы и Прообразы

Пусть $f : A \to B$ — функция, $A_1 \subset A$, $B_1 \subset B$.

- **Образ множества A_1 ($f(A_1)$):** Множество значений B, полученных из элементов A_1:

$$f(A_1) \stackrel{\text{def}}{=} \{b \in B \mid \exists a \in A_1(b = f(a))\}.$$

- **Прообраз множества B_1 ($f^{-1}(B_1)$):** Множество аргументов A, отображающихся в B_1:

$$f^{-1}(B_1) \stackrel{\text{def}}{=} \{a \in A \mid \exists b \in B_1(b = f(a))\}.$$

Теорема: Переход к образам ($F : \mathbf{2^A} \to \mathbf{2^B}$) и прообразам ($F^{-1} : \mathbf{2^B} \to \mathbf{2^A}$) также являются функциями.

Суперпозиция функций

Композиция функций называется **суперпозицией** и обозначается $\circ$ (как и композиция отношений).

Определение Если $f : A \to B$ и $g : B \to C$, то суперпозиция $g \circ f$ — это функция $g \circ f : A \to C$, определяемая как:

$$(g \circ f)(x) \stackrel{\text{def}}{=} g(f(x)).$$

Теорема: Суперпозиция функций является функцией:

$$f : A \to B \wedge g : B \to C \implies g \circ f : A \to C.$$

Представление функций в программах

Массивы (Табулирование) Если область отправления A **конечна и не очень велика**, функция представляется **массивом** (array[A] of B).

- Значение функции $f(a)$ вычисляется как $M[a]$ (обращение по индексу).
- **Эффективность:** Вычисление значения происходит за $O(1)$ (константное время).
- Функции нескольких аргументов ($f(a_1, \ldots, a_n)$) представляются **многомерными массивами**.

Процедуры (Алгоритмическое представление) Если множество A **велико или бесконечно**, функция представляется **процедурой** (блоком кода), которая вычисляет значение b по заданному аргументу a.

- В языках программирования такие процедуры также называются **функциями** (ключевое слово `function`).
- Свойство функциональности (однозначность) обеспечивается оператором `return` (возврат единственного значения).

10 Отношение эквивалентности. Классы эквивалентности. Фактор-множества. Ядро функционального отношения и множества уровня.

Отношение эквивалентности

Определение Отношение $\approx \subset M^2$ на множестве M называется **отношением эквивалентности**, если оно одновременно:

1. **Рефлексивно:** $\forall x \in M (x \approx x)$.
2. **Симметрично:** $\forall x, y \in M (x \approx y \implies y \approx x)$.
3. **Транзитивно:** $\forall x, y, z \in M (x \approx y \wedge y \approx z \implies x \approx z)$.

Примеры

- Равенство чисел ($\mathbb{R}^2$), равенство множеств ($\mathbf{2}^{\mathbf{M}^2}$).
- Равномощность множеств ($\mathbf{2}^{\mathbf{M}^2}$).

Классы эквивалентности и Фактор-множество

Класс эквивалентности Пусть $\approx$ — отношение эквивалентности на M. **Классом эквивалентности** для элемента $x \in M$ называется подмножество элементов M, эквивалентных x:

$$[x]_\approx \stackrel{\text{def}}{=} \{y \in M \mid y \approx x\}.$$

Свойства классов эквивалентности

- **Непустота:** $\forall a \in M ([a] \neq \emptyset)$. (Следует из рефлексивности).
- **Равенство:** $a \approx b \implies [a] = [b]$.
- **Дизъюнктность:** $a \not\approx b \implies [a] \cap [b] = \emptyset$.

Теорема о разбиении Если $\approx$ — отношение эквивалентности на M, то классы эквивалентности $[x]_\approx$ образуют **разбиение** множества M. И обратно, всякое разбиение множества M определяет отношение эквивалентности, классами которого являются блоки разбиения.

Фактор-множество Если R — отношение эквивалентности на M, то **фактор-множеством** M относительно R называется множество всех классов эквивалентности по R:

$$M/R \stackrel{\text{def}}{=} \{[x]_R \mid x \in M\}.$$

Фактор-множество является подмножеством булеана: $M/R \subset \mathbf{2^M}$.

Отождествление Функция $\mathrm{nat}_R : M \to M/R$, определяемая как $\mathrm{nat}_R(x) \stackrel{\text{def}}{=} [x]_R$, называется **отождествлением** (естественной сюръекцией).

Ядро функционального отношения

Ядро функционального отношения Всякое функциональное отношение (функция) $f : A \to B$ имеет **ядро**, которое является отношением на области определения $\mathrm{Dom}\ f$:

$$\ker f \stackrel{\text{def}}{=} f \circ f^{-1} \subset (\mathrm{Dom}\ f)^2.$$

Ядро $\ker f$ связывает два элемента $a_1, a_2 \in A$ тогда и только тогда, когда они имеют **одинаковые значения** при отображении f:

$$a_1 \ker f a_2 \iff f(a_1) = f(a_2).$$

Теорема Ядро функционального отношения $\ker f$ является **отношением эквивалентности** на его области определения $\mathrm{Dom}\ f$.

Множества уровня Множества, на которые разбивает $\mathrm{Dom}\ f$ ядро $\ker f$, являются классами эквивалентности $\ker f$. Эти классы называются **множествами уровня** (или **слоями**) функции f.

$$\mathrm{Dom}\ f/\ker f = \{[a]_{\ker f} \mid a \in \mathrm{Dom}\ f\}.$$

Каждый класс $[a]_{\ker f}$ состоит из всех элементов, которые отображаются функцией f в одно и то же значение $f(a)$.

$$[a]_{\ker f} = \{x \in \mathrm{Dom}\ f \mid f(x) = f(a)\}.$$

Множества уровня функции f образуют разбиение области определения $\mathrm{Dom}\ f$.

11 Отношение порядка. Минимальные элементы. Верхние и нижние границы. Монотонность. Вполне упорядоченные множества. Диаграммы Хассе. Примеры.

Отношение порядка

Определение Отношение $\prec$ на множестве M называется **отношением порядка**, если оно одновременно:

1. **Антисимметрично:** $\forall a, b \in M(a \prec b \wedge b \prec a \implies a = b)$.
2. **Транзитивно:** $\forall a, b, c \in M(a \prec b \wedge b \prec c \implies a \prec c)$.

Типы порядка

- **Нестрогий порядок** ($\preceq$): Отношение порядка, которое **рефлексивно** ($\forall a : a \preceq a$).
- **Строгий порядок** ($\prec$): Отношение порядка, которое **антирефлексивно** ($\forall a : \neg a \prec a$).
- **Линейный порядок** (Полный): Отношение порядка, которое **линейно** ($\forall a, b : a = b \vee a \prec b \vee b \prec a$).
- **Частичный порядок:** Отношение порядка, которое **не является линейным**.

Границы и специальные элементы

Минимальные и максимальные элементы *(В представленном фрагменте нет явного определения минимальных элементов, но они подразумеваются в разделе о вполне упорядоченных множествах.)*

Пусть M — частично упорядоченное множество.

- Элемент $m \in M$ называется **минимальным**, если $\neg\exists x \in M(x \prec m)$.
- Элемент $m \in M$ называется **максимальным**, если $\neg\exists x \in M(m \prec x)$.

Верхние и нижние границы Пусть $X \subset M$ — подмножество упорядоченного множества M.

- Элемент $m \in M$ — **Нижняя граница** X, если $\forall x \in X(m \preceq x)$.
- Элемент $m \in M$ — **Верхняя граница** X, если $\forall x \in X(x \preceq m)$.

Инфимум (inf X): Наибольшая нижняя граница множества X. **Супремум (sup X):** Наименьшая верхняя граница множества X.

Монотонные функции

Пусть A и B — упорядоченные множества с отношением $\preceq$, и $f : A \to B$.

Монотонно возрастающая: Если $a_1 \preceq a_2 \implies f(a_1) \preceq f(a_2)$.

Строго монотонно возрастающая: Если $a_1 \prec a_2 \implies f(a_1) \prec f(a_2)$.

Монотонно убывающая: Если $a_1 \preceq a_2 \implies f(a_2) \preceq f(a_1)$.

Строго монотонно убывающая: Если $a_1 \prec a_2 \implies f(a_2) \prec f(a_1)$.

Монотонные функции — это монотонно возрастающие или убывающие функции.

Вполне упорядоченные множества

Определение Частично упорядоченное множество X называется **вполне упорядоченным**, если **любое его непустое подмножество** имеет **минимальный элемент**.

Следствие: Вполне упорядоченное множество всегда **линейно упорядочено**, поскольку для любых двух элементов $a, b \in X$, подмножество $\{a, b\}$ должно иметь минимальный элемент, что означает, что $a \preceq b$ или $b \preceq a$.

Диаграммы Хассе

Диаграмма Хассе — это графическое представление конечного **частично упорядоченного множества** $(M, \preceq)$, где:

1. Элементы M представлены узлами.
2. Элементы, связанные отношением рефлексивности и транзитивности, **не отображаются** (транзитивное сокращение).
3. Если $a \prec b$ и нет c такого, что $a \prec c \prec b$, то b располагается выше a и соединяется с ним линией.

Диаграмма Хассе наглядно показывает **непосредственное следование** элементов в порядке.

12 Характеристические функции множеств и отношений. Функции максимума и минимума.

Характеристическая функция множества

Я ХУЙ ЗНАЕТ НОРМ НЕ НОРМ. Я НЕ НАШЕЛ (найдете киньте ISSUE)

Определение Пусть U — универсум (конечное или бесконечное множество), и A — любое его подмножество ($A \subset U$). **Характеристическая функция** χ_A множества A — это тотальная функция $\chi_A : U \to \{0, 1\}$, которая принимает значение 1, если элемент принадлежит A, и 0, если не принадлежит:

$$\chi_A(x) \stackrel{\text{def}}{=} \begin{cases} 1, & \text{если } x \in A \\ 0, & \text{если } x \notin A \end{cases}$$

Характеристическая функция устанавливает взаимно-однозначное соответствие между элементами булеана $\mathbf{2^U}$ и множеством функций $U \to \{0, 1\}$.

Характеристическая функция операций над множествами Пусть A и B — подмножества U. Операции над множествами выражаются через логические операции над их характеристическими функциями:

- **Дополнение** ($\overline{A}$): $\chi_{\overline{A}}(x) = 1 - \chi_A(x)$.
- **Пересечение** ($A \cap B$): $\chi_{A \cap B}(x) = \chi_A(x) \wedge \chi_B(x) = \chi_A(x) \cdot \chi_B(x)$.
- **Объединение** ($A \cup B$): $\chi_{A \cup B}(x) = \chi_A(x) \vee \chi_B(x) = \chi_A(x) + \chi_B(x) - \chi_A(x) \cdot \chi_B(x)$.
- **Симметрическая разность** ($A \Delta B$): $\chi_{A \Delta B}(x) = \chi_A(x) \oplus \chi_B(x)$ ($\oplus$ — исключающее "ИЛИ").

Характеристическая функция бинарного отношения

Определение Пусть A и B — множества, и R — бинарное отношение между ними ($R \subset A \times B$). **Характеристическая функция отношения** χ_R — это тотальная функция $\chi_R : A \times B \to \{0, 1\}$, которая определена на декартовом произведении и принимает значение 1, если пара принадлежит R, и 0, если не принадлежит:

$$\chi_R(a, b) \stackrel{\text{def}}{=} \begin{cases} 1, & \text{если } (a, b) \in R \quad (\text{т.е., } aRb) \\ 0, & \text{если } (a, b) \notin R \end{cases}$$

Для конечных множеств A и B характеристическая функция отношения эквивалентна его **булевой матрице** (см. Билет 7), где $\mathbf{R}[i,j] = \chi_R(a_i, b_j)$.

Функции максимума и минимума

Функции максимума (max) и минимума (min) используются для определения кратностей элементов при теоретико-множественных операциях над **мультимножествами** (см. Билет 5).

Функция максимума (для мультимножеств) Пусть $\mathcal{A}$ и $\mathcal{B}$ — мультимножества, и α_x, β_x — кратности элемента x в $\mathcal{A}$ и $\mathcal{B}$ соответственно. **Кратность элемента x в $\mathcal{A} \cup \mathcal{B}$:**

$$\mathrm{count}_{\mathcal{A}\cup\mathcal{B}}(x) = \max(\alpha_x, \beta_x).$$

Определение max**:** Для двух элементов a, b в упорядоченном множестве M (например, $\mathbb{N}$ или $\mathbb{R}$):

$$\max(a,b) \stackrel{\text{def}}{=} \begin{cases} a, & \text{если } a \geq b \\ b, & \text{если } a < b \end{cases}$$

Функция минимума (для мультимножеств) **Кратность элемента x в $\mathcal{A} \cap \mathcal{B}$:**

$$\mathrm{count}_{\mathcal{A}\cap\mathcal{B}}(x) = \min(\alpha_x, \beta_x).$$

Определение min**:** Для двух элементов a, b в упорядоченном множестве M:

$$\min(a,b) \stackrel{\text{def}}{=} \begin{cases} a, & \text{если } a \leq b \\ b, & \text{если } a > b \end{cases}$$

ЗАМЕЧАНИЕ: Функции max и min также используются для определения **верхних** (супремум, sup) и **нижних** (инфимум, inf) границ в частично упорядоченных множествах (см. Билет 11).

13 Алгебры. Носители и сигнатура. Замыкания и подалгебры. Система образующих. Свойства операций. Примеры.

Алгебры, Носители и Сигнатура

Определение Операции **n-арная (n-местная) операция** φ на множестве M — это всюду определённая (тотальная) функция $\varphi : M^n \to M$.

- Для бинарных операций ($\varphi : M \times M \to M$) используется инфиксная форма записи, например, $a * b$.

Алгебраическая структура (Универсальная алгебра) **Алгебра** $\mathcal{A}$ — это множество M вместе с набором операций $\Sigma = \{\varphi_1, \ldots, \varphi_m\}$, где $\varphi_i : M^{n_i} \to M$.

$$\mathcal{A} = (M; \Sigma) = (M; \varphi_1, \ldots, \varphi_m).$$

- **Носитель (Основа)** M: Основное множество, на котором определены операции.
- **Сигнатура** Σ: Множество всех операций $\{\varphi_1, \ldots, \varphi_m\}$.
- **Тип:** Вектор арностей $(n_1, \ldots, n_m)$.

Замыкания и Подалгебры

Замкнутое подмножество Подмножество носителя $X \subset M$ называется **замкнутым** относительно операции φ (с арностью n), если:

$$\forall x_1, \ldots, x_n \in X \implies \varphi(x_1, \ldots, x_n) \in X.$$

Множество X замкнуто относительно сигнатуры Σ, если оно замкнуто относительно всех $\varphi \in \Sigma$.

Подалгебра Алгебра $\mathcal{X} = (X; \Sigma_X)$ называется **подалгеброй** алгебры $\mathcal{A} = (M; \Sigma)$, если:

1. $X \subset M$.
2. X **замкнуто** относительно всех операций $\varphi_i \in \Sigma$.
3. Σ_X — это ограничения операций Σ на X.

Примеры Подалгебр

- Кольцо целых чисел $(\mathbb{Z}; +, \cdot)$ является подалгеброй поля рациональных чисел $(\mathbb{Q}; +, \cdot)$.
- Алгебра полиномов $\mathcal{P}_x$ является подалгеброй алгебры гладких функций $\mathcal{F}$ по операции дифференцирования.

Система образующих

Определение Подмножество $M' \subset M$ называется **системой образующих** алгебры $(M; \Sigma)$, если M является наименьшим замкнутым подмножеством, содержащим M'.

Алгебра, порождённая M' (обозначается $[M']_\Sigma$), равна M.

Конечно-порождённая алгебра: Алгебра, имеющая конечную систему образующих.

Пример Алгебра натуральных чисел $(\mathbb{N}; +)$ конечно-порождённая, так как имеет систему образующих $M' = \{1\}$.

Свойства бинарных операций

Пусть $\mathcal{A} = (M; \Sigma)$, $a, b, c \in M$, и $\circ, \bullet \in \Sigma$ — бинарные операции.

1. **Ассоциативность** ($\circ$):
$$(a \circ b) \circ c = a \circ (b \circ c).$$
2. **Коммутативность** ($\circ$):
$$a \circ b = b \circ a.$$
3. **Дистрибутивность** ($\circ$ относительно $\bullet$ слева):
$$a \circ (b \bullet c) = (a \circ b) \bullet (a \circ c).$$
4. **Поглощение** ($\circ$ поглощает $\bullet$):
$$(a \circ b) \bullet a = a.$$
5. **Идемпотентность** ($\circ$):
$$a \circ a = a.$$

14 Виды морфизмов. Гомоморфизм, изоморфизм. Примеры.

Гомоморфизм

Определение Пусть $\mathcal{A} = (A; \Sigma_{\mathcal{A}})$ и $\mathcal{B} = (B; \Sigma_{\mathcal{B}})$ — две алгебры **одного типа** (т.е., сигнатуры имеют одинаковое количество операций с одинаковыми арностями). Функция $f : A \to B$ называется **гомоморфизмом** из $\mathcal{A}$ в $\mathcal{B}$, если она **согласована с операциями** φ_i и ψ_i (где ψ_i соответствует φ_i):

$$\forall i \in \{1, \ldots, m\} : f(\varphi_i(a_1, \ldots, a_{n_i})) = \psi_i(f(a_1), \ldots, f(a_{n_i})).$$

Образно говоря, гомоморфизм «уважает» операции: результат операции над аргументами в A и последующее отображение в B равносильно отображению аргументов в B и последующей операции в B.

Коммутативная диаграмма Условие гомоморфизма для одной бинарной операции φ может быть записано через суперпозицию:

$$f \circ \varphi_{\mathcal{A}} = \varphi_{\mathcal{B}} \circ (f \times f).$$

Виды гомоморфизмов Термин **морфизм** используется как собирательное понятие для всех следующих видов отображений:

- **Эндоморфизм:** Гомоморфизм $f : \mathcal{A} \to \mathcal{A}$ (отображение алгебры на саму себя).
- **Мономорфизм:** Гомоморфизм, который является **инъекцией**.
- **Эпиморфизм:** Гомоморфизм, который является **сюръекцией**.

Пример Гомоморфизма Пусть $\mathcal{A} = (\mathbb{N}; +)$ и $\mathcal{B} = (\mathbb{N}_{10}; +_{10})$ (сложение по модулю 10). Функция $f(a) = a \bmod 10$ является гомоморфизмом из $\mathcal{A}$ в $\mathcal{B}$.

$$f(a_1 + a_2) = (a_1 + a_2) \bmod 10$$

$$f(a_1) +_{10} f(a_2) = (a_1 \bmod 10 + a_2 \bmod 10) \bmod 10$$

Обе стороны равны, что доказывает гомоморфизм.

Изоморфизм

Определение **Изоморфизм** $f : \mathcal{A} \to \mathcal{B}$ — это гомоморфизм, который является **биекцией** (одновременно инъекцией и сюръекцией). Если между алгебрами $\mathcal{A}$ и $\mathcal{B}$ существует изоморфизм, они называются **изоморфными** ($\mathcal{A} \cong \mathcal{B}$).

Свойства Изоморфизма

- **Обратный изоморфизм:** Если $f : \mathcal{A} \to \mathcal{B}$ — изоморфизм, то обратная функция $f^{-1} : \mathcal{B} \to \mathcal{A}$ также является изоморфизмом.
- **Эквивалентность:** Отношение изоморфизма ($\cong$) на множестве однотипных алгебр является **отношением эквивалентности**.
- **Автоморфизм:** Изоморфизм $f : \mathcal{A} \to \mathcal{A}$ называется **автоморфизмом**.

Значение Изоморфизма Изоморфные алгебры имеют **одинаковую структуру** с точки зрения алгебраических свойств. Любое свойство, выраженное в сигнатуре Σ, верное в $\mathcal{A}$, автоматически верно и в $\mathcal{B}$. Это позволяет изучать алгебраические структуры с **точностью до изоморфизма**.

Примеры Изоморфизма

1. Алгебра положительных вещественных чисел по умножению изоморфна алгебре всех вещественных чисел по сложению:

$$\mathcal{A} = (\mathbb{R}^+; \cdot) \cong \mathcal{B} = (\mathbb{R}; +).$$

   Изоморфизм задается функцией $f(x) = \ln(x)$.

2. Алгебра натуральных чисел по сложению изоморфна алгебре четных натуральных чисел по сложению:

$$\mathcal{A} = (\mathbb{N}; +) \cong \mathcal{B} = (\{2k \mid k \in \mathbb{N}\}; +).$$

   Изоморфизм задается функцией $f(k) = 2k$.

15 Алгебры с одной операцией. Полугруппы. Моноиды. Подстановки, композиция подстановок. Две теоремы. Примеры.

Полугруппы

Определение **Полугруппа** — это алгебра $\mathcal{S} = (M; *)$ с одной ассоциативной бинарной операцией:

$$\forall a, b, c \in M : a * (b * c) = (a * b) * c.$$

Если в полугруппе существует система образующих из одного элемента, она называется **циклической** (пример: $(\mathbb{N}; +)$ с образующей $\{1\}$).

Определяющие соотношения Если элементы полугруппы представляются как слова в алфавите образующих A, то равенства слов $\alpha = \beta$, определяющие один и тот же элемент, называются **определяющими соотношениями**.

- **Теорема (Маркова — Поста):** Существует полугруппа, в которой проблема распознавания равенства слов **алгоритмически неразрешима**.

Моноиды и подстановки

Определение моноида **Моноид** — это полугруппа с нейтральным элементом (единицей) e:

$$\exists e \in M \ (\forall a \in M : a * e = e * a = a).$$

Подстановки (в смысле замены переменных) По Новикову, **подстановкой** называется множество пар $\sigma = \{t_i//v_i\}$, где v_i — переменные, а t_i — термы.

- **Композиция подстановок** $\sigma_1 \circ \sigma_2$ — это новая подстановка, полученная последовательным применением замен. Множество подстановок образует моноид, где единица — тождественная подстановка $\{v_i//v_i\}$.

Две теоремы о моноидах

Теорема 1 (Единственность единицы) Единица моноида единственна.

Доказательство: Пусть существуют две единицы e_1 и e_2. Тогда:

1. $e_1 * e_2 = e_1$ (так как e_2 — единица).
2. $e_1 * e_2 = e_2$ (так как e_1 — единица).

Из (1) и (2) следует, что $e_1 = e_2$. $\square$

Теорема 2 (О представлении) Всякий моноид $\mathcal{M}$ над множеством M изоморфен некоторому моноиду преобразований над M.

Доказательство: Пусть $\mathcal{M} = (M; *)$ — моноид. Построим моноид преобразований $\mathcal{M}' = (F; \circ)$, где $F := \{f_m : M \to M \mid f_m(x) := x * m\}$.

1. $\mathcal{M}'$ — моноид, так как суперпозиция функций ассоциативна, f_e — тождественная функция ($f_e(x) = x * e = x$), и F замкнуто относительно $\circ$:

$$f_{a*b}(x) = x * (a * b) = (x * a) * b = f_a(x) * b = f_b(f_a(x)) = (f_a \circ f_b)(x).$$

2. Построим отображение $h : M \to F$, где $h(m) := f_m$.

3. h — гомоморфизм: $h(a * b) = f_{a*b} = f_a \circ f_b = h(a) \circ h(b)$.

4. h — биекция:

   - Сюръекция по построению множества F.
   - Инъекция: так как $f_a(e) = e * a = a$ и $f_b(e) = e * b = b$, то из $a \neq b$ следует $f_a \neq f_b$.

Следовательно, h — изоморфизм. $\square$

Примеры

1. $(\mathbb{N}_0; +)$ — моноид натуральных чисел с нулем.

2. A^* — моноид слов в алфавите A (включая пустое слово e).

3. $(\mathbb{Z}; \cdot)$ — моноид целых чисел по умножению (единица — 1).

4. Множество всех функций $f : M \to M$ относительно суперпозиции.

16 Алгебры с одной операцией. Группы. Группа перестановок.

Определение группы

Определение **Группа** — это моноид, в котором для каждого элемента a существует обратный элемент a^{-1} такой, что:

$$\forall a \in G\ (\exists a^{-1} \in G : a * a^{-1} = a^{-1} * a = e).$$

Здесь e — единица моноида, а операция $*$ часто называется умножением.

Основные понятия

- **Порядок группы:** Мощность носителя $|G|$.
- **Абелева группа:** Группа, в которой операция коммутативна ($\forall a, b : a * b = b * a$). В таких группах часто используют аддитивную запись: операция $+$, нейтральный элемент 0, обратный $-a$.

Три теоремы о свойствах групп

Теорема 1 (Единственность обратного) В группе обратный элемент a^{-1} для любого элемента a единственен.

Пусть существуют два обратных элемента a^{-1} и b для элемента a. Тогда $a * a^{-1} = a^{-1} * a = e$ и $a * b = b * a = e$. Имеем: $a^{-1} = a^{-1} * e = a^{-1} * (a * b) = (a^{-1} * a) * b = e * b = b$. Следовательно, $a^{-1} = b$.

Теорема 2 (Соотношения в группе) В любой группе выполняются следующие свойства:

1. $(a * b)^{-1} = b^{-1} * a^{-1}$.
2. Закон сокращения слева: $a * b = a * c \implies b = c$.
3. Закон сокращения справа: $b * a = c * a \implies b = c$.
4. Двойной обратный: $(a^{-1})^{-1} = a$.

1. Проверим $(a*b)*(b^{-1}*a^{-1}) = a*(b*b^{-1})*a^{-1} = a*e*a^{-1} = a*a^{-1} = e$. Аналогично для умножения слева. 2. Если $a*b = a*c$, то умножим обе части слева на a^{-1}: $a^{-1} * (a * b) = a^{-1} * (a * c) \implies (a^{-1} * a) * b = (a^{-1} * a) * c \implies e * b = e * c \implies b = c$. 3. Аналогично пункту 2, умножением на a^{-1} справа. 4. По определению $a^{-1} * a = e$, значит a является обратным для a^{-1}. Так как обратный единственен, $(a^{-1})^{-1} = a$.

Теорема 3 (Разрешимость уравнений) В группе можно однозначно решить уравнение $a * x = b$.

Решение существует: подставим $x = a^{-1} * b$. Тогда $a*(a^{-1}*b) = (a*a^{-1})*b = e*b = b$. Решение единственно: если x_1 и x_2 — решения, то $a*x_1 = b$ и $a*x_2 = b$. Значит $a * x_1 = a * x_2$, и по закону сокращения $x_1 = x_2$.

Группа перестановок

Определение Биективная функция $f : X \to X$ называется **перестановкой**. На конечном множестве перестановку удобно задавать таблицей из двух строк, называемой **подстановкой**:

$$\sigma = \begin{pmatrix} 1 & 2 & \dots & n \\ f(1) & f(2) & \dots & f(n) \end{pmatrix}$$

Множество всех перестановок n-элементного множества образует **симметрическую группу** S_n.

Свойства S_n

- **Операция:** Суперпозиция функций (результат применения f, а затем g обозначается fg).
- **Единица:** Тождественная перестановка $e(x) = x$.
- **Обратный элемент:** Обратная функция f^{-1}, таблицу которой можно получить, поменяв строки исходной таблицы местами (и отсортировав верхнюю).
- **Порядок:** $|S_n| = n!$.

Примеры групп

1. $(\mathbb{Z}; +)$ — абелева группа целых чисел. $e = 0, a^{-1} = -a$.
2. $(\mathbb{Q}^+; \cdot)$ — абелева группа положительных рациональных чисел. $e = 1, (m/n)^{-1} = n/m$.
3. $(2^M; \Delta)$ — булеан с операцией симметрической разности. $e = \varnothing, a^{-1} = a$.
4. Группа невырожденных матриц $n \times n$ относительно умножения.

17 Алгебры с двумя операциями. Кольца, поля (3 теоремы). Области целостности. Примеры.

Кольца

Определение **Кольцо** — это множество M с двумя бинарными операциями $+$ (сложение) и $*$ (умножение), удовлетворяющее следующим аксиомам:

1. **Абелева группа по сложению:** ассоциативность, коммутативность, существование нуля (0) и противоположного элемента $(-a)$.
2. **Полугруппа по умножению:** ассоциативность $(a*(b*c) = (a*b)*c)$.
3. **Дистрибутивность** умножения относительно сложения: $a*(b+c) = a*b+a*c$ и $(a+b)*c = a*c+b*c$.

Виды колец

- **Коммутативное:** если $a*b = b*a$.
- **Кольцо с единицей:** если существует $1 \in M$, такой что $a*1 = 1*a = a$ (моноид по умножению).

Теорема 1 (Свойства кольца) В любом кольце выполняются соотношения: 1. $0*a = a*0 = 0$. 2. $a*(-b) = (-a)*b = -(a*b)$. 3. $(-a)*(-b) = a*b$. 4. $-a = a*(-1)$ (в кольце с единицей).

Доказательство (пункт 1): $0*a = (0+0)*a = 0*a+0*a$. Прибавив к обеим частям $-(0*a)$, получим $0 = 0*a$. $\square$

Области целостности

Делители нуля Если в кольце для ненулевых x, y выполняется $x*y = 0$, то x и y называются **делителями нуля**. *Пример:* В машинной арифметике $(\mathbb{Z}_{2^{16}})$ имеем $256*256 = 2^{16} = 0$, хотя $256 \neq 0$.

Определение **Область целостности** — это коммутативное кольцо с единицей, не имеющее делителей нуля. *Пример:* $(\mathbb{Z}; +, *)$ — область целостности.

Теорема 2 (Закон сокращения) В кольце закон сокращения $(a*b = a*c \implies b = c$ при $a \neq 0)$ выполняется тогда и только тогда, когда в кольце нет делителей нуля.

Доказательство: $a*b = a*c \iff a*b-a*c = 0 \iff a*(b-c) = 0$. Если делителей нуля нет и $a \neq 0$, то $b-c = 0 \implies b = c$. Если есть делитель

нуля $x*y = 0$ $(x, y \neq 0)$, то $x*y = x*0$, но $y \neq 0$, т.е. сокращение невозможно. $\square$

Поля

Определение **Поле** — это коммутативное кольцо с единицей $(1 \neq 0)$, в котором каждый ненулевой элемент обратим по умножению. *Образно говоря:* Поле — это абелева группа по сложению и $M \setminus \{0\}$ — абелева группа по умножению.

Теорема 3 (Свойство поля) Всякое поле является областью целостности (т.е. не имеет делителей нуля).

Доказательство: Пусть $a * b = 0$ и $a \neq 0$. Так как мы в поле, существует a^{-1}. Умножим слева: $a^{-1} * (a * b) = a^{-1} * 0 \implies (a^{-1} * a) * b = 0 \implies 1 * b = 0 \implies b = 0$. Значит, произведение ненулевых элементов не может быть нулем. $\square$

Примеры

1. $(\mathbb{Z}; +, *)$ — коммутативное кольцо, область целостности, но **не поле** (нет обратных).

2. $(\mathbb{Q}; +, *)$, $(\mathbb{R}; +, *)$ — поля.

3. $(\mathbb{Z}_n; +, *)$ — кольцо вычетов. Является полем тогда и только тогда, когда n — простое число.

4. Машинная арифметика $(\mathbb{Z}_{2^{16}})$ — кольцо с делителями нуля (не область целостности).

18 Конечные алгебры и машинная арифметика. Двоичный смещенный код. «Арифметический круг». Двоичная и логические арифметики. Примеры.

Конечные алгебры в ЭВМ

Машинная арифметика — это реализация кольца $(\mathbb{Z}_n; +, *)$, где $n = 2^k$, а k — разрядность процессора (16, 32, 64 бита). В отличие от бесконечных целых чисел $\mathbb{Z}$, машинная арифметика конечна и имеет специфические свойства (наличие делителей нуля, переполнение).

Для визуализации операций в $\mathbb{Z}_{2^k}$ используется **арифметический круг**. Элементы располагаются по кругу от 0 до $2^k - 1$.

- Сложение соответствует движению по часовой стрелке.
- Вычитание — против часовой стрелки.
- **Переполнение** возникает, когда результат операции «перескакивает» через границу $2^k - 1 \to 0$.

В знаковой арифметике круг делится пополам: первая половина $[0, 2^{k-1}-1]$ — положительные числа, вторая половина $[2^{k-1}, 2^k - 1]$ — отрицательные числа (в дополнительном коде).

Двоичный смещенный код Для представления вещественных чисел (экспоненты) или упрощения сравнения целых используется **смещенный код**. Число x представляется как $x_{code} = x + B$, где B — фиксированное смещение (обычно $B = 2^{k-1}$).

- Это позволяет представлять отрицательные числа натуральными, сохраняя порядок (большему числу соответствует больший код).

Двоичная и логические арифметики Новиков разделяет операции над машинными словами на два типа:

1. **Двоичная арифметика:** Операции в кольце $(\mathbb{Z}_{2^k}; +, *)$. Здесь учитываются переносы между разрядами.
2. **Логические арифметики:** Поразрядные операции $(\wedge, \vee, \oplus, \neg)$. В этой структуре каждый бит независим, и перенос не возникает. Это алгебра $(\mathbb{B}^k; \wedge, \vee, \neg)$.

Пример В 8-битной арифметике (Z_{256}): $200 + 100 = 300 \pmod{256} = 44$. (Двоичная арифметика). 11001000_2 AND $01100100_2 = 01000000_2$. (Логическая арифметика).

19 «Малая» и «большая» конечные арифметики. Примеры.

Ф.А. Новиков выделяет два уровня формализации вычислений в ЭВМ, называя их «малой» и «большой» арифметиками.

«Малая» конечная арифметика

Это арифметика на уровне **отдельных битов**.

- **Носитель:** Булево множество $\mathbb{B} = \{0, 1\}$.
- **Операции:** Логическое И ($\wedge$), ИЛИ ($\vee$), НЕ ($\neg$), Исключающее ИЛИ ($\oplus$).
- **Суть:** Описывает работу логических вентилей процессора. Здесь нет понятия «числа» в привычном смысле, только логические значения.

«Большая» конечная арифметика

Это арифметика на уровне **машинных слов** (регистров).

- **Носитель:** Множество всех возможных значений регистра $M = \{0, 1, \ldots, 2^k - 1\}$.
- **Операции:** Сложение и умножение по модулю 2^k.
- **Суть:** Описывает поведение целых чисел в программах. Это кольцо $(\mathbb{Z}_{2^k}; +, *)$.

Связь между ними

«Большая» арифметика реализуется через «малую». Например, операция сложения в «большой» арифметике строится из логических операций «малой» арифметики (сумматоров), учитывающих **перенос** (carry).

Примеры

1. **Малая:** Вычисление четности бита через $a \oplus b$.
2. **Большая:** Работа счетчика цикла, который при достижении значения $2^k - 1$ обнуляется при следующем шаге (свойство кольца вычетов).
3. **Логический сдвиг:** Операция в большой арифметике, которая эффективно соответствует умножению на 2 в кольце, но реализуется как перемещение битов.

20 Векторное пространство. Линейные комбинации. Базис и размерность. Модули. Примеры.

Векторное пространство

Определение Пусть $\mathcal{F} = (F; +, \cdot)$ — поле (элементы — скаляры), а $\mathcal{V} = (V; +)$ — абелева группа (элементы — векторы). $\mathcal{V}$ называется **векторным пространством** над полем $\mathcal{F}$, если определена операция умножения вектора на скаляр $F \times V \to V$, удовлетворяющая аксиомам ($\forall a, b \in F, \forall x, y \in V$):

1. $(a + b)x = ax + bx$
2. $a(x + y) = ax + ay$
3. $(a \cdot b)x = a(bx)$
4. $1x = x$

Теорема (Свойства нулей) 1. $0x = \mathbf{0}$ (умножение нулевого скаляра на вектор дает нуль-вектор). 2. $a\mathbf{0} = \mathbf{0}$ (умножение скаляра на нуль-вектор дает нуль-вектор).

Доказательство: 1. $0x = (1-1)x = 1x - 1x = x - x = \mathbf{0}$. 2. $a\mathbf{0} = a(x - x) = ax - ax = \mathbf{0}$. $\square$

Линейные комбинации и независимость

Определения

- **Линейная комбинация** векторов x_i: сумма вида $\sum_{i=1}^{n} a_i x_i$, где $a_i \in F$.
- **Линейная независимость:** Множество векторов $\{x_1, \ldots, x_k\}$ называется линейно независимым, если равенство $\sum a_i x_i = \mathbf{0}$ выполняется только при всех $a_i = 0$. В противном случае множество **линейно зависимо**.

Теорема Линейно независимое множество векторов не содержит нуль-вектора.
Доказательство: Если $x_1 = \mathbf{0}$, то можно взять $a_1 = 1$, а остальные $a_i = 0$. Тогда $1 \cdot \mathbf{0} + 0 \cdot x_2 + \cdots = \mathbf{0}$, при этом $a_1 \neq 0$, что означает линейную зависимость. $\square$

Базис и размерность

Определение

- **Порождающее множество (система образующих):** Подмножество S, такое что любой вектор пространства можно представить как линейную комбинацию элементов из S.

- **Базис:** Линейно независимое порождающее множество.

- **Размерность:** Количество элементов в базисе (обозначается $\dim V$).

Теорема 1 (Единственность разложения) Каждый вектор имеет единственное представление в данном базисе. *Доказательство:* Если $x = \sum a_i E_i$ и $x = \sum b_i E_i$, то $\sum (a_i - b_i) E_i = \mathbf{0}$. В силу линейной независимости базиса $a_i - b_i = 0$, т.е. $a_i = b_i$. $\square$

Теорема 2 (О размере) Размер любого линейно независимого множества не превосходит размера базиса ($m \leq n$).

Модули

Если в определении векторного пространства заменить поле $\mathcal{F}$ на произвольное **кольцо** (не обязательно поле), то полученная структура называется **модулем** над кольцом. Главное отличие: в модуле не всегда существует базис и не для каждого элемента есть обратный скаляр.

Примеры

1. F^n — пространство кортежей длины n над полем F.

2. **Булеан:** $(2^M, \Delta)$ является векторным пространством над полем $\mathbb{Z}_2$ (где $1X = X, 0X = \varnothing$).

3. V_X — пространство формальных сумм $\sum \lambda_x x$, «натянутое» на конечное множество X.

4. Пространство функций $\Phi : X \to F$ с покомпонентными операциями.

21 Решётки и их виды. Частичный порядок в решётке. Булевы алгебры. Примеры.

Определения и аксиомы решётки

Определение **Решётка** — это множество M с двумя бинарными операциями $\cap$ (пересечение) и $\cup$ (объединение), удовлетворяющими следующим аксиомам:

1. **Идемпотентность:** $a \cap a = a$, $a \cup a = a$.
2. **Коммутативность:** $a \cap b = b \cap a$, $a \cup b = b \cup a$.
3. **Ассоциативность:** $(a \cap b) \cap c = a \cap (b \cap c)$, $(a \cup b) \cup c = a \cup (b \cup c)$.
4. **Поглощение:** $(a \cap b) \cup a = a$, $(a \cup b) \cap a = a$.

Виды решёток

- **Дистрибутивная решётка:** если операции распределены друг относительно друга:

$$a \cup (b \cap c) = (a \cup b) \cap (a \cup c), \quad a \cap (b \cup c) = (a \cap b) \cup (a \cap c).$$

- **Ограниченная решётка:** если существуют **нуль** 0 ($\forall a : 0 \cap a = 0$) и **единица** 1 ($\forall a : 1 \cup a = 1$).
- **Решётка с дополнением:** ограниченная решётка, в которой для каждого a существует a', такой что $a \cap a' = 0$ и $a \cup a' = 1$.

Частичный порядок в решётке

Определение порядка В любой решётке можно ввести отношение частичного порядка $\preceq$:

$$a \preceq b \overset{\text{def}}{\iff} a \cap b = a \quad (\text{что эквивалентно } a \cup b = b).$$

Связь с гранями Решётку можно определить и через порядок. Если в частично упорядоченном множестве для любых двух элементов существуют:

- **Нижняя грань:** $x = \inf(a, b)$ (наибольшая из нижних границ).
- **Верхняя грань:** $y = \sup(a, b)$ (наименьшая из верхних границ).

То такое множество является решёткой, где $a \cap b = \inf(a, b)$ и $a \cup b = \sup(a, b)$.

Свойства дополнения

В ограниченной **дистрибутивной** решётке дополнение обладает важными свойствами:

1. **Единственность:** у каждого элемента только одно дополнение.
2. **Инволютивность:** $a'' = a$.
3. **Законы де Моргана:** $(a \cap b)' = a' \cup b'$ и $(a \cup b)' = a' \cap b'$.

Булевы алгебры

Определение **Булева алгебра** — это дистрибутивная ограниченная решётка, в которой для каждого элемента существует дополнение. Она объединяет в себе все свойства операций $\cap, \cup, '$ и констант $0, 1$.

Примеры

1. **Булеан:** $(2^M; \cap, \cup, \overline{A})$ — множество всех подмножеств с операциями пересечения, объединения и дополнения.
2. **Логика:** $(\{0, 1\}; \wedge, \vee, \neg)$ — классическая двузначная логика. Порядком здесь является импликация.
3. **Делители числа:** Пусть P — множество произведений различных простых чисел из набора T. Тогда $(P; \text{НОД}, \text{НОК}, \text{ДОП})$ — булева алгебра. Здесь $0 = 1$, $1 = \prod_{p \in T} p$, а порядок — отношение «делится на».

22 Матроиды. Максимальные независимые подмножества, базисы матроида. Примеры матроидов. Жадный алгоритм.

Определение матроида

Определение **Матроидом** называется пара $M = (E, \mathcal{I})$, где E — конечное множество носителя ($|E| = n$), а $\mathcal{I} \subseteq 2^E$ — семейство его подмножеств, называемых **независимыми**, для которых выполняются три аксиомы:

- **M1:** $\emptyset \in \mathcal{I}$ (пустое множество независимо).
- **M2:** Если $A \in \mathcal{I}$ и $B \subset A$, то $B \in \mathcal{I}$ (подмножество независимого множества независимо).
- **M3 (Аксиома замены):** Если $A, B \in \mathcal{I}$ и $|B| = |A| + 1$, то существует элемент $e \in B \setminus A$, такой что $A \cup \{e\} \in \mathcal{I}$.

Пример Семейство линейно независимых векторов любого векторного пространства является матроидом. Это исторический прообраз понятия матроида.

Максимальные независимые подмножества и базисы

Определение Пусть $X \subseteq E$. **Максимальным независимым подмножеством** множества X называется такое $Y \subseteq X$, что $Y \in \mathcal{I}$, и не существует другого $Z \in \mathcal{I}$, такого что $Y \subset Z \subseteq X$. Множество таких подмножеств обозначается X'.

Аксиома M4 и равномощность Важнейшее свойство матроида: во всяком подмножестве X все максимальные независимые подмножества имеют одинаковую мощность (**аксиома M4**).

$$\forall X \subseteq E \ (\forall Y, Z \in X' \implies |Y| = |Z|).$$

Новиков доказывает, что при выполнении M1 и M2 аксиомы M3 и M4 эквивалентны.

Базисы Максимальные независимые подмножества всего носителя E называются **базисами** матроида. Согласно свойству M4, все базисы матроида имеют одинаковую мощность (ранг матроида).

Жадный алгоритм

Постановка задачи Задана весовая функция $w : E \to \mathbb{R}^+$. Нужно найти независимое множество $X \in \mathcal{I}$ с максимальным суммарным весом $w(X) = \sum_{e \in X} w(e)$.

Алгоритм 1. Упорядочить элементы E по убыванию весов: $w(e_1) \geq w(e_2) \geq \cdots \geq w(e_n)$. 2. Инициализировать $X = \emptyset$. 3. Для i от 1 до n: если $X \cup \{e_i\} \in \mathcal{I}$, то включить e_i в X.

Теорема о жадном алгоритме Жадный алгоритм находит оптимальное решение (множество максимального веса) для любой весовой функции w **тогда и только тогда**, когда структура $(E, \mathcal{I})$ является матроидом.

Примеры и контрпримеры

1. **Матричный матроид:** E — столбцы матрицы, $\mathcal{I}$ — линейно независимые наборы столбцов. Жадный алгоритм здесь находит базис с максимальным весом.

2. **Пример "Один элемент из столбца":** Если задача — выбрать по одному элементу из каждого столбца матрицы с макс. суммой, то это матроид (жадный алгоритм работает).

3. **Контрпример "Один из столбца и строки":** Если нужно выбрать элементы так, чтобы из каждой строки и каждого столбца было не более одного элемента, то жадный алгоритм может выдать неоптимальный результат.

Значение Матроиды позволяют сразу определить, применим ли эффективный жадный алгоритм ($O(n)$) для решения сложной экстремальной задачи, или же структура задачи требует более тяжелых методов (динамическое программирование, перебор).

23 Булевы функции. Функции одной и двух переменных, функции трех переменных. Существенные и фиктивные переменные.

Определение булевой функции

Определение Булевой функцией (функцией алгебры логики) от n переменных называется функция $f : E_2^n \to E_2$, где $E_2 = \{0, 1\}$.

- **Количество функций:** Для n переменных существует 2^{2^n} различных булевых функций.
- **Задание таблицы:** Функция задается таблицей истинности из 2^n строк. В учебнике Новикова принят **установленный порядок** перечисления наборов — лексикографический (по возрастанию двоичных чисел).

Существенные и фиктивные переменные

Определение Переменная x_i называется **существенной** для функции $f(x_1, \ldots, x_n)$ если существует такой набор значений остальных переменных $(\alpha_1, \ldots, \alpha_{i-1}, \alpha_{i+1}, \ldots, \alpha_n$ на котором изменение значения x_i меняет значение функции:

$$f(\alpha_1, \ldots, 0, \ldots, \alpha_n) \neq f(\alpha_1, \ldots, 1, \ldots, \alpha_n).$$

В противном случае переменная называется **несущественной (фиктивной)**.

Удаление и введение переменных Булевы функции рассматриваются с точностью до несущественных переменных.

- **Удаление:** Если x_i фиктивна, таблицу можно сократить вдвое, удалив столбец x_i и повторяющиеся строки.
- **Введение:** Любую функцию $f(x_1, \ldots, x_n)$ можно рассматривать как функцию от большего числа переменных, добавив фиктивные.

Элементарные булевы функции

Функции одной переменной ($n = 1$) Существует $2^{2^1} = 4$ функции:

1. **Константа 0** (Нуль).
2. **Тождественная функция** $f(x) = x$.
3. **Отрицание** $f(x) = \neg x$ (инверсия).
4. **Константа 1** (Единица).

Функции двух переменных ($n = 2$) Существует $2^{2^2} = 16$ функций. Основные из них:

- $x \wedge y$ — **Конъюнкция** (логическое умножение).
- $x \vee y$ — **Дизъюнкция** (логическое сложение).
- $x \oplus y$ — **Сложение по модулю 2** (XOR, исключающее ИЛИ).
- $x \to y$ — **Импликация** (если x, то y).
- $x \equiv y$ — **Эквивалентность**.
- $x|y$ — **Штрих Шеффера** (И-НЕ).
- $x \downarrow y$ — **Стрелка Пирса** (ИЛИ-НЕ).

Функции трех переменных ($n = 3$) Существует $2^{2^3} = 256$ функций. Они обычно не имеют специальных названий и представляются в виде **формул** через функции 1 и 2 переменных.

24 Формулы. Реализация функций формулами, интерпретация формул. Таблицы истинности. Равносильные формулы.

Формулы и их реализация

Определение формулы **Формула** над множеством функций F (называемым базисом) — это цепочка символов, построенная по следующим правилам:

1. Любая переменная x_i является формулой.
2. Если $\mathcal{J}_1, \ldots, \mathcal{J}_n$ — формулы, а $f \in F$ — функция арности n, то выражение $f(\mathcal{J}_1, \ldots, \mathcal{J}_n)$ является формулой.

В записи часто используют инфиксную форму ($a \vee b$ вместо $\vee(a, b)$) и приоритет операций ($\neg, \&, \vee, \rightarrow$), чтобы опускать лишние скобки.

Интерпретация (Алгоритм Eval) Каждой формуле $\mathcal{J}$ соответствует булева функция. Процесс вычисления значения формулы на наборе значений переменных называется **интерпретацией**. **Рекурсивный алгоритм Eval:**

- Если $\mathcal{J}$ — переменная x_i, вернуть её заданное значение.
- Если $\mathcal{J} = f(\mathcal{J}_1, \ldots, \mathcal{J}_n)$, то сначала вычислить значения всех подформул $\mathcal{J}_i$, а затем применить к ним операцию f.

Если формула $\mathcal{J}$ при интерпретации на всех наборах выдает те же значения, что и функция f, говорят, что **формула реализует функцию** ($\text{func}\ \mathcal{J} = f$).

Таблицы истинности и равносильность

Построение таблицы Для определения функции, которую реализует формула, строится таблица истинности. Для этого формула разбивается на подформулы (операции), и для каждой вычисляется промежуточный столбец. *Пример:* Для формулы $\mathcal{J} = (x_1 \wedge x_2) \vee (\neg x_1 \wedge x_2)$ итоговый столбец совпадет со столбцом функции x_2.

Равносильные формулы Две формулы $\mathcal{J}_1$ и $\mathcal{J}_2$ называются **равносильными** ($\mathcal{J}_1 \equiv \mathcal{J}_2$), если они реализуют одну и ту же функцию. Отношение равносильности является отношением эквивалентности.

Основные равносильности (Законы булевой алгебры) Новиков выделяет 10 базовых групп (проверяются таблицами истинности):

1. **Идемпотентность:** $a \vee a = a, a \wedge a = a$.
2. **Коммутативность:** $a \vee b = b \vee a, a \wedge b = b \wedge a$.
3. **Ассоциативность:** $a \vee (b \vee c) = (a \vee b) \vee c$.
4. **Поглощение:** $(a \wedge b) \vee a = a, (a \vee b) \wedge a = a$.
5. **Дистрибутивность:** $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$.
6. **Константы:** $a \vee 1 = 1, a \wedge 0 = 0$.
7. **Нейтральные элементы:** $a \vee 0 = a, a \wedge 1 = a$.
8. **Двойное отрицание:** $\neg\neg a = a$.
9. **Законы де Моргана:** $\neg(a \wedge b) = \neg a \vee \neg b, \neg(a \vee b) = \neg a \wedge \neg b$.
10. **Дополнение:** $a \vee \neg a = 1, a \wedge \neg a = 0$.

Подстановка и замена

Правило подстановки Если в равносильной формуле заменить все вхождения некоторой переменной x на одну и ту же формулу $\mathcal{A}$, то полученные формулы останутся равносильными. Это позволяет выводить сложные равносильности из простых.

Правило замены Если в формуле $\mathcal{A}$ заменить некоторую подформулу $\mathcal{B}$ на равносильную ей подформулу $\mathcal{C}$ ($\mathcal{B} \equiv \mathcal{C}$), то полученная формула $\mathcal{A}'$ будет равносильна исходной ($\mathcal{A} \equiv \mathcal{A}'$).

Примеры

1. Формула $((x_1 \wedge x_2) \to x_1)$ реализует **константу 1** (тавтология).
2. Формула $((x_1 \wedge x_2) + x_1) + x_2$ реализует **дизъюнкцию** $x_1 \vee x_2$.

25 Алгебра булевых функций и булевы алгебры. Алгебра Линденбаума-Тарского. Эквивалентные преобразования. Разложение Шеннона.

Эквивалентные преобразования формул

Для преобразования формул с сохранением их смысла (реализуемой функции) используются два основных правила.

Правило подстановки (Теорема 1) Если две формулы равносильны ($\mathcal{J}_1 = \mathcal{J}_2$), то при замене **всех** вхождений некоторой переменной x на одну и ту же формулу $\mathcal{S}$ результат останется равносильным:

$$\mathcal{J}_1\{\mathcal{S}//x\} = \mathcal{J}_2\{\mathcal{S}//x\}.$$

Важно: условие замены **всех** вхождений критично (иначе тождество может нарушиться).

Правило замены (Теорема 2) Если в формуле $\mathcal{J}$ заменить некоторую подформулу $\mathcal{S}_1$ на равносильную ей формулу $\mathcal{S}_2$, то полученная формула будет равносильна исходной:

$$\mathcal{S}_1 = \mathcal{S}_2 \implies \mathcal{J}\{\mathcal{S}_1\} = \mathcal{J}\{\mathcal{S}_2/\mathcal{S}_1\}.$$

Алгебра булевых функций

Операции $\vee, \wedge, \neg$ можно рассматривать не только на значениях $\{0, 1\}$, но и на самих функциях $f, g \in P_n$. Результатом операции $f \vee g$ будет новая функция, значение которой на каждом наборе равно дизъюнкции значений исходных функций.

Алгебра Линденбаума-Тарского Это формализация «алгебры логики» как математической структуры.

- Рассмотрим множество всех формул над некоторым базисом.
- Введем на нем отношение эквивалентности $\equiv$ (равносильность).
- Множество классов эквивалентности формул относительно операций $\vee, \wedge, \neg$ образует **булеву алгебру** (см. билет 21).

Это позволяет нам преобразовывать сложные логические выражения, используя законы булевой алгебры (дистрибутивность, законы де Моргана и т.д.), как обычные алгебраические уравнения.

Разложение Шеннона (Разложение по переменным)

Любую булеву функцию можно разложить по любой её переменной x_i. Это фундаментальная теорема, позволяющая сводить функцию n переменных к функциям $n-1$ переменной.

Теорема (Разложение Шеннона) Для любой функции $f(x_1, \ldots, x_n)$ справедливо:

$$f(x_1, \ldots, x_n) = (x_i \wedge f(x_1, \ldots, 1, \ldots, x_n)) \vee (\neg x_i \wedge f(x_1, \ldots, 0, \ldots, x_n)).$$

Пояснение:

- $f(x_1, \ldots, 1, \ldots, x_n)$ — это функция, полученная подстановкой константы 1 вместо x_i.
- $f(x_1, \ldots, 0, \ldots, x_n)$ — это функция, полученная подстановкой константы 0 вместо x_i.

Значение разложения 1. Позволяет систематически строить формулы для любых функций (через СДНФ). 2. Является основой для работы алгоритмов упрощения булевых функций и построения бинарных диаграмм решений (BDD).

Примеры эквивалентных преобразований

1. $x \to y = \neg x \vee y$ (выражение импликации через базис $\{\neg, \vee\}$).
2. $\neg(x \wedge y) = \neg x \vee \neg y$ (закон де Моргана).
3. $x \vee (x \wedge y) = x$ (закон поглощения).

26 Двойственность в математике. Принцип двойственности. Представление булевых функций в программах.

Двойственная функция

Определение Пусть $f(x_1, \ldots, x_n)$ — булева функция. Функция f^*, определяемая как:

$$f^*(x_1, \ldots, x_n) \stackrel{\text{def}}{=} \neg f(\neg x_1, \neg x_2, \ldots, \neg x_n),$$

называется **двойственной** к функции f.

Свойства

1. **Инволютивность:** $f^{**} = f$. Это означает, что отношение «быть двойственной» симметрично.

2. **Табличное правило:** Чтобы получить таблицу истинности для f^*, нужно инвертировать все значения в таблице функции f (и значения аргументов, и значения самой функции) и переставить строки в установленный порядок. Проще говоря: столбец значений f^* читается как инвертированный столбец f, записанный снизу вверх.

3. **Самодвойственность:** Функция называется самодвойственной, если $f^* = f$. Примеры: x, $\neg x$, медиана $M(x, y, z)$. Конъюнкция и дизъюнкция не самодвойственны ($x \wedge y \leftrightarrow x \vee y$).

Реализация двойственной функции и Принцип двойственности

Теорема (о реализации) Если функция φ реализована формулой $\mathcal{J}(f_1, \ldots, f_m)$, то двойственная функция φ^* реализуется формулой $\mathcal{J}^*(f_1^*, \ldots, f_m^*)$, где каждая подфункция заменена на двойственную ей.

Принцип двойственности Пусть формула $\mathcal{J}$ в базисе $F = \{f_1, \ldots, f_m\}$ реализует функцию f. Тогда формула $\mathcal{J}^*$, полученная из $\mathcal{J}$ заменой каждой функции f_i на двойственную ей f_i^*, реализует двойственную функцию f^*.

- **Следствие:** Если $\mathcal{J}_1 = \mathcal{J}_2$ — равносильность, то $\mathcal{J}_1^* = \mathcal{J}_2^*$ — также равносильность.

- *Пример:* Из закона де Моргана $\neg(x \wedge y) = \neg x \vee \neg y$ по принципу двойственности автоматически следует $\neg(x \vee y) = \neg x \wedge \neg y$.

Представление булевых функций в программах

В программировании булевы функции представляются несколькими основными способами:

1. **Вектор значений (Truth Table):** Функция от n переменных представляется как целое число или массив битов длиной 2^n. Например, функция $x \wedge y$ — это вектор 0001_2 (число 1), а $x \vee y$ — 0111_2 (число 7). Это самый быстрый способ для функций с малым n (до 5–6 переменных).

2. **Аналитическое (формульное):** Функция хранится как дерево выражений (Expression Tree) или цепочка символов. Используется в системах символьных вычислений и компиляторах.

3. **Бинарные решающие диаграммы (BDD):** Графовое представление, позволяющее компактно хранить и быстро сравнивать функции от большого числа переменных.

4. **Битовые операции:** Процессорные инструкции (AND, OR, NOT, XOR) позволяют вычислять булеву функцию одновременно для 32 или 64 наборов данных, если они упакованы в машинные слова. Это называется **битовым параллелизмом**.

27 Разложение булевых функций по переменным. Нормальные формы. КНФ, ДНФ, СКНФ, СДНФ. Сокращенные дизъюнктивные формы.

Разложение по переменным (Шеннон)

Любая функция может быть представлена через свои значения на частных наборах:

$$f(x_1, \ldots, x_n) = (x_i \wedge f(x_1, \ldots, 1, \ldots, x_n)) \vee (\neg x_i \wedge f(x_1, \ldots, 0, \ldots, x_n)).$$

Если применить это разложение последовательно по всем n переменным, мы придем к понятию совершенных нормальных форм.

Совершенные нормальные формы

Определение СДНФ **Совершенная дизъюнктивная нормальная форма** — это реализация функции в виде дизъюнкции элементарных конъюнкций, каждая из которых содержит **все** переменные (в прямом или инверсном виде).

$$f(x_1, \ldots, x_n) = \bigvee_{\sigma : f(\sigma)=1} (x_1^{\sigma_1} \wedge \cdots \wedge x_n^{\sigma_n}).$$

Процедура: Берем все строки таблицы истинности, где $f = 1$. Для каждой строки пишем конъюнкцию: если в строке $x_i = 1$, пишем x_i; если $x_i = 0$, пишем $\neg x_i$.

Определение СКНФ **Совершенная конъюнктивная нормальная форма** — это реализация функции в виде конъюнкции элементарных дизъюнкций, содержащих все переменные.

$$f(x_1, \ldots, x_n) = \bigwedge_{\sigma : f(\sigma)=0} (x_1^{\neg\sigma_1} \vee \cdots \vee x_n^{\neg\sigma_n}).$$

Процедура: Берем строки, где $f = 0$. Пишем дизъюнкцию: если в строке $x_i = 0$, пишем x_i; если $x_i = 1$, пишем $\neg x_i$.

Теорема о единственности Всякая булева функция (кроме констант 0 и 1 в соответствующих случаях) имеет **единственную** СДНФ и СКНФ (при установленном лексикографическом порядке переменных и слагаемых).

Эквивалентные преобразования к СДНФ

Любую формулу можно привести к СДНФ за 7 шагов: 1. **Элиминация:** замена всех операций $(\to, \oplus, \equiv)$ на $\{\vee, \wedge, \neg\}$. 2. **Протаскивание отрицаний:** по законам де Моргана $\neg$ доводится до переменных. 3. **Расскрытие скобок:** использование дистрибутивности конъюнкции. 4. **Удаление нулей:** убирание конъюнкций вида $(x \wedge \neg x)$. 5. **Приведение подобных:** удаление лишних вхождений по идемпотентности. 6. **Расщепление:** добавление недостающих переменных по правилу $A = (A \wedge x) \vee (A \wedge \neg x)$. 7. **Сортировка:** упорядочивание переменных внутри конъюнкций и самих конъюнкций.

ДНФ и Сокращенная форма

ДНФ (Дизъюнктивная нормальная форма) Это дизъюнкция элементарных конъюнкций произвольного ранга. В отличие от СДНФ, здесь в слагаемых могут отсутствовать некоторые переменные.

Сокращенная ДНФ Это ДНФ, состоящая из суммы всех **простых импликант** функции.

- **Импликанта:** конъюнкция K, такая что $K \to f = 1$.
- **Простая импликанта:** импликанта, из которой нельзя удалить ни одной переменной, чтобы она осталась импликантой.

Сокращенная ДНФ получается из СДНФ путем всех возможных операций **склеивания** $((A \wedge x) \vee (A \wedge \neg x) = A)$ и последующего поглощения.

28 Минимизация булевых функций. Минимальные и сокращенные ДНФ. Геометрическая интерпретация.

Постановка задачи минимизации

Булева функция может быть реализована бесконечным множеством равносильных формул. Задача минимизации состоит в поиске самой «простой» формулы.

Критерии простоты:

- Наименьшее количество вхождений переменных.
- Наименьшее количество логических операций.

Наиболее изучена задача минимизации в классе **дизъюнктивных нормальных форм (ДНФ)**.

Дизъюнктивные формы

Определения

- **Элементарная конъюнкция** K — это конъюнкция переменных или их отрицаний, в которую каждая переменная входит не более одного раза.
- **Ранг конъюнкции** $|K|$ — количество переменных, входящих в неё.
- **ДНФ** — это дизъюнкция элементарных конъюнкций: $\bigvee_{i=1}^{k} K_i$.

Число различных ДНФ от n переменных равно 2^{3^n}, что делает полный перебор для минимизации практически невозможным уже при $n > 5$.

Геометрическая интерпретация

Новиков предлагает рассматривать наборы значений переменных как вершины **n-мерного единичного гиперкуба** E^n.

- **Вершина** (0-мерная грань) соответствует набору из n констант (элементарной конъюнкции ранга n).
- **Ребро** (1-мерная грань) соответствует конъюнкции ранга $n-1$ (две вершины, различающиеся в одном разряде).
- **k-мерная грань** соответствует конъюнкции ранга $n-k$. Чем меньше переменных в конъюнкции, тем большую область («облако» единиц) в кубе она покрывает.

Булева функция задается подмножеством вершин гиперкуба, на которых она равна 1. Задача минимизации превращается в задачу **покрытия всех «единичных» вершин минимальным числом граней максимальной размерности**.

Сокращенные и минимальные ДНФ

Простая импликанта Элементарная конъюнкция K называется **импликантой** функции f, если $K = 1 \implies f = 1$. Импликанта называется **простой**, если при удалении из неё любой переменной она перестает быть импликантой (геометрически: это грань максимальной размерности, целиком состоящая из единиц функции).

Сокращенная ДНФ ДНФ, состоящая из дизъюнкции **всех** простых импликант функции, называется **сокращенной**. *Способ получения:* 1. Взять СДНФ функции. 2. Применять закон **склеивания**: $(x \wedge \mathcal{A}) \vee (\neg x \wedge \mathcal{A}) = \mathcal{A}$. 3. Применять закон **поглощения**: $\mathcal{A} \vee (\mathcal{A} \wedge \mathcal{B}) = \mathcal{A}$.

Минимальная ДНФ (МДНФ) Это ДНФ, содержащая минимальное суммарное число вхождений переменных. МДНФ всегда строится из некоторого подмножества простых импликант, входящих в сокращенную ДНФ. *Алгоритм (вкратце):* Из сокращенной ДНФ нужно выбрать такой минимальный набор конъюнкций, который в сумме «покрывает» все те же единицы, что и исходная функция.

Пример Для функции $f(x, y) = (x \wedge y) \vee (x \wedge \neg y) \vee (\neg x \wedge y)$: 1. Склеиваем $(x \wedge y)$ и $(x \wedge \neg y) \to x$. 2. Склеиваем $(x \wedge y)$ и $(\neg x \wedge y) \to y$. 3. Сокращенная ДНФ: $x \vee y$. Она же в данном случае будет минимальной.

29 Полиномы Жегалкина (три способа получения). Линейное представление булевых функций.

Полином Жегалкина

Определение **Полиномом Жегалкина** называется выражение (полином) над полем $\mathbb{Z}_2$, то есть формула в базисе $\{\oplus, \wedge, 1\}$. В такой формуле отсутствуют отрицания и дизъюнкции, а операции подчиняются законам арифметики по модулю 2:

$$x \oplus x = 0, \quad x \oplus 0 = x, \quad x \wedge x = x, \quad x \wedge 1 = x.$$

Теорема о единственности Всякая булева функция единственным образом (с точностью до порядка слагаемых) представляется в виде полинома Жегалкина.

Три способа получения полинома

1. **Метод эквивалентных преобразований:** Берется любая формула (например, СДНФ) и в ней производятся замены:

   - $\neg x \to x \oplus 1$
   - $x \vee y \to x \oplus y \oplus (x \wedge y)$

   Затем раскрываются скобки по дистрибутивности, и слагаемые с четным числом вхождений взаимно уничтожаются ($A \oplus A = 0$).

2. **Метод неопределенных коэффициентов:** Полином ищется в общем виде: $f(x_1, x_2) = a_0 \oplus a_1 x_1 \oplus a_2 x_2 \oplus a_{12} x_1 x_2$. Подставляя все 2^n наборов значений переменных, получаем систему линейных уравнений относительно коэффициентов $a_i \in \{0, 1\}$. Система всегда имеет треугольный вид и легко решается.

3. **Метод треугольника (сумматорный метод):** Строится таблица, в первом столбце которой записываются значения функции из таблицы истинности. Каждый следующий столбец получается путем сложения по модулю 2 соседних элементов предыдущего столбца. Верхние элементы полученных столбцов будут являться коэффициентами полинома Жегалкина при соответствующих конъюнкциях в лексикографическом порядке.

Линейное представление

Булева функция называется **линейной**, если её полином Жегалкина не содержит произведений переменных (т.е. его степень ≤ 1):

$$f(x_1, \ldots, x_n) = a_0 \oplus a_1 x_1 \oplus \cdots \oplus a_n x_n.$$

Линейные функции играют ключевую роль в теории кодирования и криптографии.

30 Дифференцирование булевых функций.

Определение производной **Производной** булевой функции $f(x_1, \ldots, x_n)$ по переменной x_i называется функция, которая показывает, меняет ли исходная функция свое значение при изменении только переменной x_i:

$$\frac{\partial f}{\partial x_i} \stackrel{\text{def}}{=} f(x_1, \ldots, x_i, \ldots, x_n) \oplus f(x_1, \ldots, \neg x_i, \ldots, x_n).$$

Эквивалентная форма через разложение Шеннона:

$$\frac{\partial f}{\partial x_i} = f(x_1, \ldots, 0, \ldots, x_n) \oplus f(x_1, \ldots, 1, \ldots, x_n).$$

Свойства дифференцирования Операция дифференцирования в булевой алгебре обладает свойствами, похожими на классический анализ:

1. **Линейность:** $\frac{\partial(f \oplus g)}{\partial x} = \frac{\partial f}{\partial x} \oplus \frac{\partial g}{\partial x}$.
2. **Производная константы:** $\frac{\partial C}{\partial x} = 0$.
3. **Производная отрицания:** $\frac{\partial(\neg f)}{\partial x} = \frac{\partial f}{\partial x}$.
4. **Правило произведения (аналог Лейбница):** $\frac{\partial(f \wedge g)}{\partial x} = \left(\frac{\partial f}{\partial x} \wedge g\right) \oplus \left(f \wedge \frac{\partial g}{\partial x}\right) \oplus \left(\frac{\partial f}{\partial x} \wedge \frac{\partial g}{\partial x}\right)$.

Связь с существенными переменными Переменная x_i является **существенной** для функции f тогда и только тогда, когда производная $\frac{\partial f}{\partial x_i}$ не является константой 0. Если производная тождественно равна нулю, переменная фиктивна.

Формула Тейлора Любую булеву функцию можно представить через значения её производных в точке (аналог разложения в ряд Тейлора), что используется в задачах синтеза и диагностики цифровых схем.

31 Деревья решений (полные и сокращенные). Синтаксические деревья. Бинарная диаграмма решений.

Синтаксические деревья

Определение **Синтаксическое дерево** — это иерархическое представление структуры **формулы**.

- Корнем является главная (внешняя) операция формулы.
- Узлами — промежуточные операции.
- Листьями — переменные или константы.

Это дерево отражает порядок вычислений (интерпретации) по алгоритму Eval: сначала вычисляются значения детей, затем — значение родителя.

Деревья решений (семантические деревья)

Полное дерево решений Таблицу истинности функции от n переменных можно представить в виде полного бинарного дерева высоты $n+1$.

- **Ярусы:** соответствуют переменным $x_1, \ldots, x_n$ в установленном порядке.
- **Дуги:** левая дуга соответствует значению 0, правая — 1.
- **Листья:** хранят значения функции f на кортеже, соответствующем пути из корня.

Сокращенное дерево решений Если у некоторого узла все листья в его поддереве имеют одно и то же значение (все 0 или все 1), то всё поддерево можно заменить этим значением. Это значительно уменьшает объём хранимых данных.

Бинарная диаграмма решений (BDD)

Определение **Бинарная диаграмма решений** получается из дерева решений путём отказа от древовидности связей (допускается несколько входящих дуг в один узел). Построение диаграммы включает три преобразования:

1. **Отождествление листьев:** Все листовые узлы, содержащие 0, сливаются в один узел «0». Аналогично — для «1».

2. **Слияние изоморфных поддиаграмм:** Если два узла одного яруса ведут в одни и те же узлы при одинаковых значениях переменной, они заменяются одним экземпляром.

3. **Исключение избыточных узлов:** Если обе дуги из узла (0 и 1) ведут в один и тот же следующий узел, этот узел удаляется, а входящие в него дуги перенаправляются напрямую к следующему узлу.

Зависимость от порядка переменных Результат построения диаграммы существенно зависит от порядка переменных. Правильный выбор порядка (например, вынесение наиболее влияющей переменной в корень) может сделать диаграмму экспоненциально компактнее. *Пример Новикова:* Функция, которая при одном порядке требует сложной диаграммы, при другом (например, y, x, z) может превратиться в простое условие ‘if y then 1 else !x‘.

Алгоритм интерпретации

Вычисление значения функции по дереву или диаграмме (Алгоритм 3.5) выполняется проходом от корня к листу:

- На каждом шаге смотрим на значение текущей переменной x_i.
- Если $x_i = 0$, переходим по левой дуге, если $x_i = 1$ — по правой.
- Когда достигнут листовой узел, возвращается его значение.

Преимущество Представление функции в виде BDD часто позволяет хранить меньше информации и производить вычисления быстрее, чем при использовании традиционных таблиц или формул.

32 Полнота. Замкнутые классы. Полные системы функций и базисы. Теорема Поста.

Функциональная полнота и замкнутость

Определения

- **Замыкание** $[F]$ системы функций F — это множество всех функций, которые можно реализовать формулами в базисе F.
- **Замкнутый класс** — это множество функций K, которое совпадает со своим замыканием ($[K] = K$). То есть суперпозиция функций из этого класса не выводит за его пределы.
- **Полная система** — это система функций F, замыкание которой совпадает со всем множеством булевых функций ($[F] = P_n$).
- **Базис** — это полная система, которая перестает быть полной при удалении любого её элемента.

Пять предполных классов Поста

Новиков выделяет пять классов функций, обладающих свойством замкнутости:

1. T_0 — **Класс функций, сохраняющих нуль:** $f(0, \ldots, 0) = 0$.
2. T_1 — **Класс функций, сохраняющих единицу:** $f(1, \ldots, 1) = 1$.
3. T^* — **Класс самодвойственных функций:** $f = f^*$.
4. $T_\leq$ — **Класс монотонных функций:** $\alpha \preceq \beta \implies f(\alpha) \leq f(\beta)$.
5. T_L — **Класс линейных функций:** функции, представимые полиномом Жегалкина степени ≤ 1.

Теорема Поста

Формулировка Система булевых функций F является **полной** тогда и только тогда, когда она не содержится целиком ни в одном из пяти замкнутых классов $T_0, T_1, T^*, T_\leq, T_L$.

Доказательство (схема)

1. **Необходимость:** Если система F целиком лежит в одном из классов (например, все функции сохраняют 0), то и любая их суперпозиция будет сохранять 0. Тогда мы не сможем получить функцию $\neg x$, так как $\neg 0 = 1$.

2. **Достаточность:** Если функции системы F «разрушают» свойства всех 5 классов, то из них можно построить базис $\{\neg, \wedge\}$:

   - С помощью функций $\notin T_0$ и $\notin T_1$ строятся **константы** 0 и 1.
   - С помощью функции $\notin T_{\leq}$ (немонотонной) и констант строится **отрицание**.
   - С помощью функции $\notin T_L$ (нелинейной), отрицания и констант строится **конъюнкция**.

Примеры полных систем

1. $\{\neg, \wedge, \vee\}$ — избыточная полная система.
2. $\{\neg, \wedge\}$ — стандартный базис.
3. $\{\neg, \vee\}$ — стандартный базис.
4. $\{|\}$ (Штрих Шеффера) — полная система из одной функции.
5. $\{\downarrow\}$ (Стрелка Пирса) — полная система из одной функции.
6. $\{x \wedge y, x \oplus y, 1\}$ — базис Жегалкина.

Принцип двойственности в полноте Если система F полна, то система двойственных ей функций F^* также является полной. Это следует из того, что любая функция h может быть выражена через F, а значит h^* выразима через F^*.

Пример проверки по таблице Поста Для системы $\{\neg, \wedge\}$:

- $\neg x$: $\notin T_0$ (т.к. $\neg 0 = 1$), $\notin T_1$ (т.к. $\neg 1 = 0$), $\notin T_{\leq}$.
- $x \wedge y$: $\in T_0, \in T_1, \in T_{\leq}$, но $\notin T^*$ и $\notin T_L$.
- **Итог:** Вместе они перекрывают все 5 классов $\implies$ система полна.

33 Логические формулы, связки и кванторы. Интерпретация, логическое следование.

Логические связки и высказывания

Высказывание — это утверждение, которое либо истинно (T), либо ложно (F). Простые высказывания обозначаются пропозициональными переменными.

Логические связки (в порядке старшинства):

- Отрицание ($\neg$): «не».
- Конъюнкция (&): «и».
- Дизъюнкция ($\vee$): «или».
- Импликация ($\rightarrow$): «если... то».

Пропозициональные формулы

Синтаксис (форма Бэкуса-Наура): $\langle$формула$\rangle ::= T \mid F \mid \langle$переменная$\rangle \mid (\neg\langle$формула$\rangle) \mid (\langle$формула$\rangle\&\langle$формула$\rangle) \mid (\langle$формула$\rangle\vee\langle$формула$\rangle) \mid (\langle$формула$\rangle \rightarrow \langle$формула$\rangle)$.

Интерпретация и классификация формул

Интерпретация I — это конкретный набор истинностных значений, приписанных переменным формулы. Значение формулы A в интерпретации I обозначается $I(A)$.

Классификация формул:

1. **Выполнимая:** истинна хотя бы при одной интерпретации.
2. **Общезначимая (тавтология):** истинна при **всех** возможных интерпретациях.
3. **Невыполнимая (противоречие):** ложна при всех возможных интерпретациях.

Теорема: Если формулы A и $A \rightarrow B$ являются тавтологиями, то формула B — также тавтология (правило Modus Ponens).

Логическое следование и эквивалентность

Логическое следование ($A \Rightarrow B$) Формула B логически следует из A, если B принимает значение T во всех интерпретациях, где A имеет значение T.

- **Теорема 3:** $P_1 \& \ldots \& P_n \Rightarrow Q$ тогда и только тогда, когда $(P_1 \& \ldots \& P_n) \to Q$ — тавтология.
- **Теорема 4:** $P_1 \& \ldots \& P_n \Rightarrow Q$ тогда и только тогда, когда $P_1 \& \ldots \& P_n \& \neg Q$ — противоречие. (Это теоретическая основа метода резолюций).

Логическая эквивалентность ($A \equiv B$) Формулы логически эквивалентны, если они являются логическими следствиями друг друга (имеют одинаковые таблицы истинности). Алгебра $(\{T, F\}; \vee, \&, \neg)$ является булевой алгеброй высказываний.

Кванторы (Логика предикатов)

Для работы с объектами и их свойствами в формулы вводятся **кванторы**:

- **Квантор всеобщности** ($\forall x$)**:** «для всех x». Формула $\forall x P(x)$ истинна, если свойство P выполняется для всех объектов предметной области.
- **Квантор существования** ($\exists x$)**:** «существует такой x, что». Формула $\exists x P(x)$ истинна, если найдется хотя бы один объект со свойством P.

Интерпретация с кванторами включает в себя: 1. Выбор непустого множества (предметной области). 2. Определение предикатов (свойств) на этом множестве. 3. Определение значений свободных переменных.

34 Фундаментальные узлы компьютерных систем и двоичные функции.

Реализация компьютерных систем базируется на том, что любая булева функция может быть воплощена в виде электронной схемы. В основе этого лежит связь между «малой» арифметикой (логические операции над битами) и «большой» арифметикой (операции над машинными словами).

Логические вентили (Базовые элементы)

Каждая элементарная булева функция реализуется физическим устройством, называемым **вентилем**:

- **Инвертор (NOT):** реализует $\neg x$.
- **Конъюнктор (AND):** реализует $x \wedge y$.
- **Дизъюнктор (OR):** реализует $x \vee y$.
- **Исключающее ИЛИ (XOR):** реализует $x \oplus y$ (основа сумматоров).

Комбинационные узлы

Узлы, значение на выходе которых зависит только от текущих значений на входе (реализация булевых функций «в чистом виде»).

1. Сумматор (Основа арифметики) Служит для сложения двоичных чисел. Строится на основе разложения суммы по разрядам.

- **Полусумматор (Half Adder):** Складывает два бита x и y.
  - Сумма: $S = x \oplus y$
  - Перенос: $C = x \wedge y$
- **Полный сумматор (Full Adder):** Складывает два бита x, y и перенос из предыдущего разряда C_{in}.
  - $S = x \oplus y \oplus C_{in}$
  - $C_{out} = (x \wedge y) \vee (C_{in} \wedge (x \oplus y))$

2. Дешифратор (Decoder) Узел, который преобразует двоичный код в сигнал на одной из выходных линий.

- Если на вход подано n бит, то выходов будет 2^n.
- Каждый выход соответствует определенной **элементарной конъюнкции** (минтерму) входных переменных.

3. Мультиплексор (Multiplexer) Узел, который выбирает один из нескольких информационных входов и передает его на единственный выход. Выбор осуществляется на основе адресного кода.

- Реализует **разложение Шеннона**: адресные линии выступают в роли переменных разложения, выбирая нужную подфункцию.

Узлы с памятью (Последовательностные)

Для реализации полноценных вычислений необходимы элементы, хранящие состояние (память).

RS-триггер Простейший элемент памяти, строящийся на обратной связи (например, на двух элементах ИЛИ-НЕ / Стрелках Пирса).

- Позволяет хранить 1 бит информации.
- В отличие от комбинационных схем, здесь значение функции зависит не только от входов R (Reset) и S (Set), но и от предыдущего значения выхода Q.

Синтез схем

Согласно теореме о функциональной полноте (билет 32), используя только вентили И, ИЛИ, НЕ (или только штрих Шеффера), можно построить узел любой сложности.

- **Прямой синтез:** По таблице истинности строится СДНФ, которая напрямую переводится в двухуровневую логическую схему (слой конъюнкторов $\rightarrow$ слой дизъюнкторов).
- **Минимизация:** Использование МДНФ (билет 28) позволяет сократить количество вентилей и входов, что критично для стоимости и скорости работы процессора.

35 Алфавит, слово и язык. Кодирование и его свойства. Методы описания множества сообщений. Алфавитное кодирование.

Алфавит, слово и язык

Определения

- **Алфавит** (A) — конечное множество символов (букв).
- **Слово** — любая конечная последовательность символов алфавита A.
- **Пустое слово** (ε) — слово, не содержащее ни одного символа. Длина $|\varepsilon| = 0$.
- **Длина слова** $|w|$ — количество символов в слове.
- **Множество слов:**
  - A^* — множество всех слов в алфавите A, включая пустое слово.
  - A^+ — множество всех непустых слов ($A^* \setminus \{\varepsilon\}$).
- **Язык** (L) — любое подмножество множества всех слов ($L \subseteq A^*$).

Методы описания множества сообщений

Сообщения — это слова некоторого языка. Описать язык можно тремя способами: 1. **Перечисление:** (только для конечных языков) $\{\text{слово}_1, \text{слово}_2, \dots\}$. 2. **Порождающие процедуры:** использование грамматик (правил вывода слов). 3. **Распознающие процедуры:** использование автоматов или регулярных выражений, которые отвечают «да/нет» на вопрос, принадлежит ли слово языку.

Кодирование и его свойства

Кодирование — это отображение f элементов одного множества (исходных символов S) в слова другого алфавита B (кодовые слова).

$$f : S \to B^*$$

Свойства кодирования:

1. **Инъективность:** разным исходным символам соответствуют разные кодовые слова.

2. **Однозначная декодируемость:** любое слово в кодовом алфавите (последовательность кодовых слов) может быть разложено на исходные символы единственным способом.

3. **Префиксное свойство (Условие Фано):** никакое кодовое слово не является началом (префиксом) другого кодового слова.

4. **Оптимальность:** минимизация средней длины кодового слова (например, код Хаффмана).

Алфавитное кодирование

Схема кодирования Пусть задан исходный алфавит $S = \{s_1, s_2, \ldots, s_n\}$ и целевой алфавит B. **Схема алфавитного кодирования** — это таблица соответствия:

- $s_1 \to w_1$
- $s_2 \to w_2$
- $\ldots$
- $s_n \to w_n$

где $w_i \in B^*$. Кодирование сообщения $s_{i_1} s_{i_2} \ldots s_{i_k}$ осуществляется простой конкатенацией кодовых слов: $w_{i_1} w_{i_2} \ldots w_{i_k}$.

Виды кодов:

- **Равномерные коды:** все кодовые слова w_i имеют одинаковую длину (пример: код ASCII, где на символ 8 бит).
- **Неравномерные коды:** длины кодовых слов различаются (пример: азбука Морзе, код Хаффмана).

Примеры

1. **Двоичное кодирование:** $A = \{a, b, c, d\}$, схема: $a \to 00, b \to 01, c \to 10, d \to 11$. Это равномерный префиксный код.

2. **Код Хаффмана:** для часто встречающихся символов назначаются короткие коды, для редких — длинные. Код является префиксным по построению.

3. **Азбука Морзе:** $s_1 \to \cdot, s_2 \to -$. Не является префиксным кодом в чистом виде (требует разделителя — паузы между буквами), поэтому строго говоря, алфавитом кодирования в Морзе является $\{\cdot, -, \text{пауза}\}$.

36 Разделимые и префиксные схемы кодирования. Неравенство Макмиллана. Кодирование с минимальной избыточностью. Цена кодирования. Примеры.

Схемы кодирования

Пусть $M = \{m_1, \ldots, m_n\}$ — алфавит сообщений, а $A = \{a_1, \ldots, a_d\}$ — кодирующий алфавит. Схема алфавитного кодирования сопоставляет каждому символу m_i слово w_i в алфавите A.

Разделимые коды Схема кодирования называется **разделимой** (однозначно декодируемой), если любое слово в алфавите A, полученное в результате кодирования некоторой последовательности сообщений, может быть расшифровано единственным способом. *Пример:* $\{0, 01\}$ — неразделим (непонятно, 01 — это одно сообщение или два: 0 и 1).

Префиксные коды (Условие Фано) Схема кодирования называется **префиксной**, если ни одно кодовое слово не является префиксом (началом) другого кодового слова.

- **Свойство:** Любой префиксный код является разделимым.
- **Преимущество:** Декодирование может осуществляться мгновенно (нам не нужно ждать конца сообщения, чтобы понять, где закончилось текущее кодовое слово).

Пример префиксного кода: $\{0, 10, 110, 111\}$.

Неравенство Макмиллана

Эта теорема устанавливает необходимое и достаточное условие существования разделимого кода с заданными длинами слов $l_1, l_2, \ldots, l_n$.

Теорема Для того чтобы существовал разделимый код в алфавите мощности d с длинами кодовых слов $l_1, l_2, \ldots, l_n$, необходимо и достаточно выполнение неравенства:

$$\sum_{i=1}^{n} d^{-l_i} \leq 1.$$

Важное следствие: Для любого разделимого кода существует префиксный код с такими же длинами слов. Поэтому на практике ограничиваются поиском префиксных кодов.

Цена и избыточность кодирования

Цена кодирования (средняя длина) Если сообщения m_i появляются с вероятностями p_i, то средней длиной (ценой) кода называется математическое ожидание длины кодового слова:

$$L = \sum_{i=1}^{n} p_i \cdot l_i.$$

Кодирование с минимальной избыточностью Задача состоит в поиске такой префиксной схемы, при которой цена L минимальна для данных вероятностей p_i.

- Теоретический предел минимальной цены задается энтропией источника H.
- **Избыточность** — это разность между средней длиной кода L и энтропией H.

Алгоритм Хаффмана (Пример)

Это жадный алгоритм построения оптимального префиксного кода. 1. Выбираются два сообщения с наименьшими вероятностями. 2. Они объединяются в новый узел с суммарной вероятностью. 3. Процесс повторяется, пока не останется один узел (корень дерева). 4. Путь от корня до листа определяет код (0 — поворот влево, 1 — вправо).

Пример Сообщения $A(0.5), B(0.25), C(0.25)$. - Объединяем B и C (сумма 0.5). - Объединяем результат с A (сумма 1.0). - Коды: $A = 0, B = 10, C = 11$. - Цена $L = 0.5 \cdot 1 + 0.25 \cdot 2 + 0.25 \cdot 2 = 1.5$ бита на символ.

37 Алгоритм Фано. Оптимальное кодирование. Алгоритм Хаффмана. Дерево Хаффмана.

Оптимальное кодирование

Постановка задачи Пусть задан алфавит $S = \{s_1, \ldots, s_n\}$ и вероятности появления каждого символа $p_1, \ldots, p_n$. Задача **оптимального кодирования** — построить такую схему префиксного кодирования, при которой **средняя длина кодового слова** будет минимальной:

$$L_{cp} = \sum_{i=1}^{n} p_i \cdot l_i \to \min,$$

где l_i — длина кодового слова для символа s_i. Основная идея: часто встречающимся символам назначаются короткие коды, а редким — длинные.

Алгоритм Фано (Шеннона-Фано)

Это «нисходящий» (top-down) метод построения кода.

Алгоритм: 1. Отсортировать символы алфавита по убыванию вероятностей. 2. Разделить алфавит на две части так, чтобы суммарные вероятности обеих частей были максимально близки друг к другу. 3. Первой части присвоить в качестве первого разряда кода «0», второй — «1». 4. Рекурсивно повторить процедуру для каждой части, пока в группах не останется по одному символу.

Особенности: Метод Фано гарантирует получение префиксного кода, но не всегда дает математически оптимальный результат (иногда суммы вероятностей нельзя разделить достаточно ровно).

Алгоритм Хаффмана

Это «восходящий» (bottom-up) метод, который всегда строит **математически оптимальный** префиксный код.

Алгоритм: 1. Выписать все символы как свободные узлы будущего дерева, указав их вероятности (веса). 2. Выбрать два узла с **наименьшими** вероятностями. 3. Создать для них родительский узел, вес которого равен сумме весов этих двух узлов. 4. Заменить два выбранных узла на новый родительский узел в списке свободных узлов. 5. Повторять шаги 2–4, пока не останется один узел — **корень дерева**.

Дерево Хаффмана

Результатом работы алгоритма является бинарное дерево, называемое **деревом Хаффмана**.

- **Листья дерева** — это исходные символы алфавита.
- **Ветви** помечаются символами «0» (обычно левое ребро) и «1» (правое).
- **Код символа** — это последовательность меток на пути от корня к соответствующему листу.

Сравнение и пример

Пример: Алфавит $\{A : 0.4, B : 0.3, C : 0.2, D : 0.1\}$.

- **Хаффман:** Складываем $D(0.1)$ и $C(0.2) \to CD(0.3)$. Затем $CD(0.3)$ и $B(0.3) \to BCD(0.6)$. Наконец $BCD(0.6)$ и $A(0.4) \to Root(1.0)$.
- **Коды:** A получит короткий код «0» (1 бит), а D — длинный «111» (3 бита).

Преимущества Хаффмана: 1. **Минимальная избыточность:** средняя длина сообщения ближе всего к энтропии источника. 2. **Префиксность:** сообщение можно декодировать «на лету» без разделителей. 3. **Эффективность:** Хаффман всегда лучше или равен методу Фано.

38 Помехоустойчивое кодирование. Кодирование с исправлением ошибок. Возможность исправления всех ошибок.

Основные понятия

Помехоустойчивое кодирование — это способ представления данных, позволяющий обнаруживать или исправлять ошибки, возникшие при передаче по каналу с шумом. Главная идея заключается во введении **избыточности**: из всех возможных кодовых слов 2^n мы используем только некоторое подмножество «разрешенных» слов. Если принятое слово не входит в это подмножество, значит, произошла ошибка.

Расстояние Хэмминга

Для анализа ошибок Новиков использует метрику в пространстве двоичных слов.

Расстояние Хэмминга $d(x, y)$ — это количество позиций (разрядов), в которых слова x и y одинаковой длины различаются.

Минимальное расстояние кода d_{min} — это минимальное из расстояний Хэмминга между всеми парами различных разрешенных кодовых слов.

Обнаружение и исправление ошибок

Способность кода бороться с помехами напрямую зависит от d_{min}:

1. **Обнаружение k ошибок:** Чтобы гарантированно заметить k ошибок, необходимо, чтобы $d_{min} \geq k + 1$. *Пример:* Код с проверкой на четность ($d_{min} = 2$) обнаруживает 1 ошибку, но не может её исправить.

2. **Исправление k ошибок:** Чтобы гарантированно исправить k ошибок, необходимо, чтобы $d_{min} \geq 2k + 1$. *Логика:* Испорченное слово должно остаться «ближе» к исходному разрешенному слову, чем к любому другому. Это называется **декодированием по принципу максимального правдоподобия**.

Геометрическая интерпретация: Сферы Хэмминга

Вокруг каждого разрешенного кодового слова можно представить «сферу» радиуса k. Если сферы радиуса k вокруг разных кодовых слов не пересекаются, то любую ошибку кратности $\leq k$ можно исправить, «притянув» испорченное слово обратно к центру сферы.

Возможность исправления всех ошибок

Ограничения Можно ли исправить **вообще все** ошибки?

- **Нет**, если количество ошибок превышает исправляющую способность кода ($k > \lfloor \frac{d_{min}-1}{2} \rfloor$). В этом случае слово может «улететь» в сферу другого кодового слова, и система совершит ложное исправление.
- **Теорема Шеннона:** Для канала с шумом существует такая скорость передачи данных $R < C$ (пропускная способность), при которой вероятность ошибки можно сделать сколь угодно малой, но не равной нулю.

Совершенные коды Это коды, в которых сферы Хэмминга не только не пересекаются, но и **плотно упаковывают** все пространство (каждое возможное слово либо является разрешенным, либо находится в сфере радиуса k ровно одного разрешенного слова). Пример: Коды Хэмминга.

Примеры

- **Код с повторением (3-кратным):** $0 \to 000, 1 \to 111$. $d_{min} = 3$. Можно обнаружить 2 ошибки или исправить 1 ошибку (мажоритарный метод: «голосование» большинством).
- **Код Хэмминга (7, 4):** К 4 информационным битам добавляется 3 контрольных. Позволяет исправлять любую одиночную ошибку.

39 Расстояния Левенштейна и Хэмминга. Кодовое расстояние схемы. Мощность кодирования.

Для обнаружения и исправления ошибок в каналах связи необходимо уметь измерять «различие» между словами. В теории кодирования для этого используются две основные метрики.

1. Расстояние Хэмминга

Определение **Расстоянием Хэмминга** $d_H(u, v)$ между двумя словами u и v **одинаковой длины** называется количество позиций (разрядов), в которых соответствующие символы различны. *Пример:* $d_H(10110, 11010) = 2$ (различия во 2-м и 3-м битах).

Свойства 1. Расстояние Хэмминга является метрикой (удовлетворяет неравенству треугольника). 2. Используется в блочном кодировании, где ошибки — это только **замены** (инверсии битов), но не вставки или удаления.

2. Расстояние Левенштейна (редакционное расстояние)

Определение **Расстоянием Левенштейна** между двумя словами (возможно, разной длины) называется минимальное количество операций **вставки, удаления и замены** одного символа, необходимых для превращения одного слова в другое.

Значение Эта метрика более универсальна, чем расстояние Хэмминга, так как она учитывает ошибки синхронизации (когда бит пропадает или появляется лишний). Вычисляется обычно с помощью динамического программирования.

3. Кодовое расстояние схемы

Определение **Кодовым расстоянием** d_{min} схемы кодирования называется **минимальное** из расстояний Хэмминга между всеми парами различных кодовых слов этой схемы:

$$d = \min_{i \neq j} d_H(w_i, w_j).$$

Роль кодового расстояния Именно кодовое расстояние определяет корректирующую способность кода:

- Для обнаружения r ошибок необходимо, чтобы $d \geq r + 1$.
- Для исправления s ошибок необходимо, чтобы $d \geq 2s + 1$.

Пример: Если $d = 3$, мы можем гарантированно обнаружить 2 ошибки или исправить 1 ошибку (правило «ближайшего соседа»).

4. Мощность кодирования

Определение **Мощностью кода** M называется количество кодовых слов в данной схеме кодирования. В равномерном коде длины n над алфавитом мощности q максимально возможная мощность — q^n. Однако для помехоустойчивости мы используем только часть этих слов, вводя **избыточность**.

Параметры кода Код обычно характеризуется тройкой (n, M, d), где:

- n — длина кодового слова.
- M — мощность (число разрешенных комбинаций).
- d — кодовое расстояние.

Информационная скорость Отношение $\frac{\log_q M}{n}$ называется скоростью кода. Она показывает, какая часть передаваемой информации является полезной. Существует фундаментальное противоречие: чем больше кодовое расстояние (надежность), тем меньше мощность кода при фиксированной длине (ниже скорость).

40 Классические коды коррекции ошибок (коды с повторением, линейные коды). Код Хэмминга.

Коды с повторением

Это простейший вид помехоустойчивого кодирования.

Принцип: Каждый бит сообщения m передается n раз. *Пример (n=3):* $0 \to 000, 1 \to 111$.

- **Расстояние:** Минимальное кодовое расстояние $d_{min} = n$.
- **Исправляющая способность:** Позволяет исправлять $k = \lfloor \frac{n-1}{2} \rfloor$ ошибок методом «голосования» (мажоритарный принцип).
- **Недостаток:** Крайне низкая скорость передачи данных (высокая избыточность). При $n = 3$ скорость $R = 1/3$.

Линейные (групповые) коды

Линейный код — это более эффективный способ внесения избыточности, основанный на линейной алгебре.

Определение Код называется **линейным**, если множество его кодовых слов образует подпространство в векторном пространстве V_n над полем $\mathbb{Z}_2$.

- **Свойство:** Сумма (по модулю 2) двух любых кодовых слов также является кодовым словом.
- **Нулевой вектор:** Нулевое слово всегда принадлежит линейному коду.
- **Расстояние:** Минимальное расстояние d_{min} линейного кода равно минимальному весу (количеству единиц) ненулевого кодового слова.

Задание кода Линейный (n, k)-код (где n — длина слова, k — число инфобит) задается: 1. **Порождающей матрицей** G**:** размером $k \times n$. Кодовое слово v получается как $v = m \cdot G$. 2. **Проверочной матрицей** H**:** размером $(n - k) \times n$. Для любого разрешенного слова v выполняется $v \cdot H^T = \mathbf{0}$.

Код Хэмминга для исправления одного замещения

Коды Хэмминга — это класс совершенных линейных кодов, которые исправляют ровно одну ошибку.

Принцип построения (на примере кода (7, 4)) Мы добавляем к 4 информационным битам ($k = 4$) 3 контрольных бита ($r = 3$), итого длина $n = 7$. 1. Контрольные биты ставятся на позиции, являющиеся степенями двойки (1, 2, 4). 2. Каждый контрольный бит отвечает за проверку на четность определенной группы позиций. 3. Группы выбираются так, чтобы в двоичной записи номера любой позиции «1» в j-м разряде означала, что эта позиция проверяется j-м контрольным битом.

Исправление ошибки (Синдром) При получении слова v' вычисляется **синдром** $S = v' \cdot H^T$.

- Если $S = \mathbf{0}$, ошибок нет.
- Если $S \neq \mathbf{0}$, то значение синдрома в двоичной системе счисления прямо указывает на **номер позиции** бита, в котором произошла ошибка (замещение).

Пример Если пришел вектор и синдром получился $110_2 = 6$, значит, нужно инвертировать 6-й бит в принятом сообщении, чтобы восстановить оригинал.

Преимущество: Высокая скорость передачи (мало избыточных бит) при гарантированном исправлении одиночных ошибок.